

Pre-training

Lecture Plan

1. Introduction to pretraining
2. Model pretraining in three ways
3. Scaling pretraining
4. Pretraining data

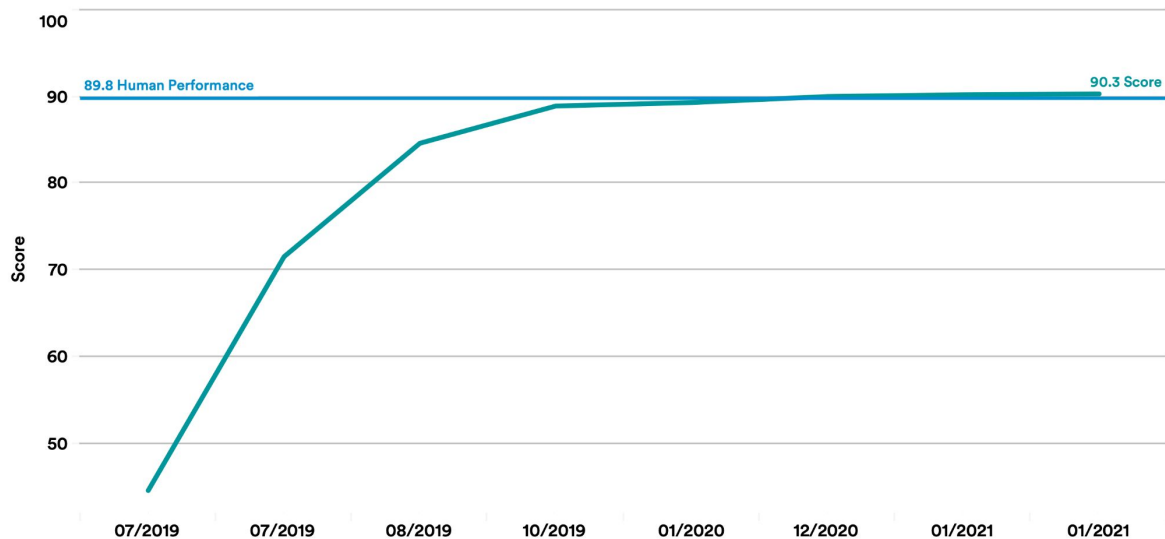
Introduction to Pretraining

The pretraining revolution

Pretraining has had a huge impact on how well NLP systems work

SUPERGLUE BENCHMARK

Source: SuperGLUE Leaderboard, 2020 | Chart: 2021 AI Index Report



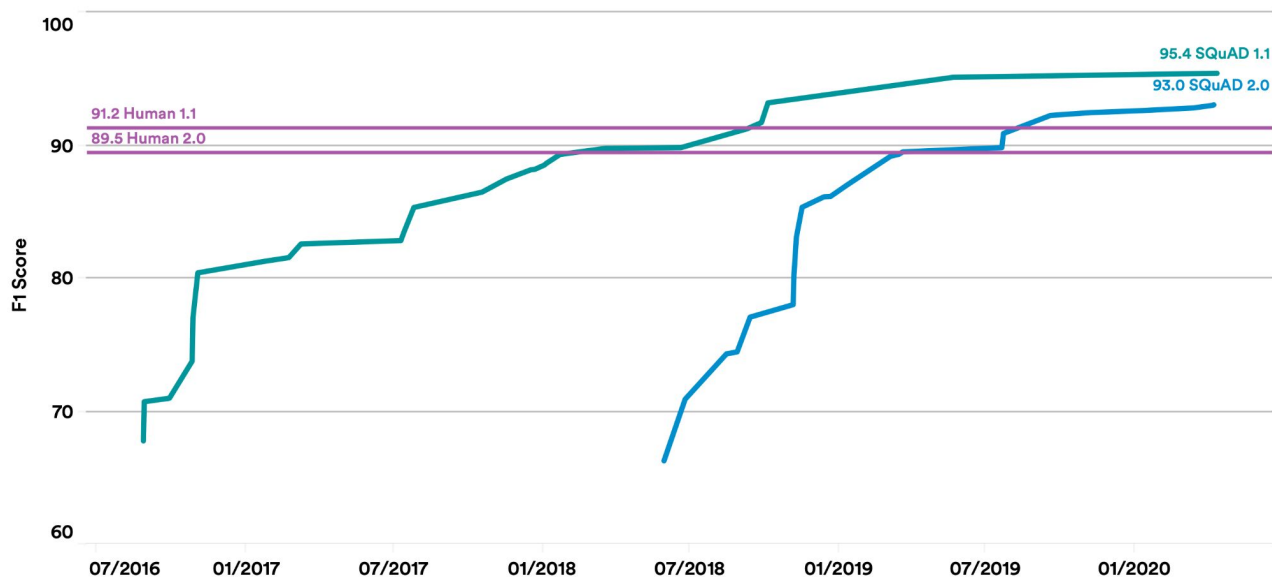
[HAI 2021 AI Index Report](#)

The pretraining revolution

Pretraining has had a huge impact on how well NLP systems work

SQUAD 1.1 and SQUAD 2.0: F1 SCORE

Source: CodaLab Worksheets, 2020 | Chart: 2021 AI Index Report



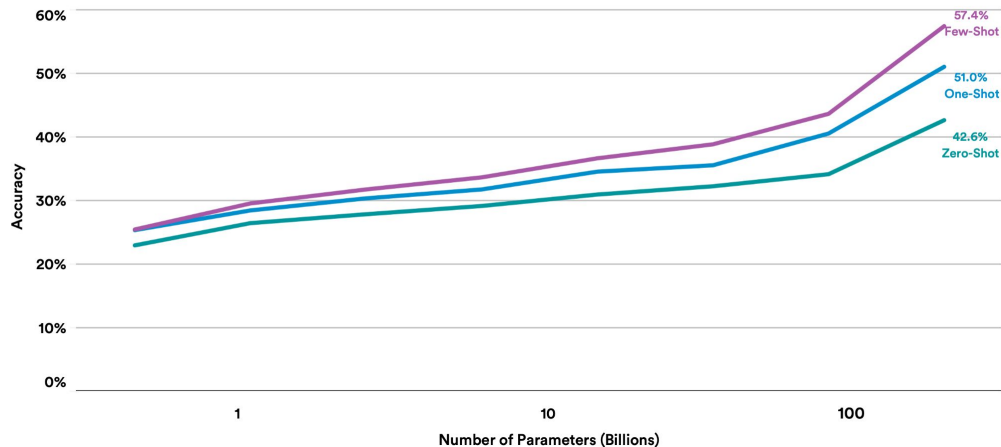
The pretraining revolution

Before: design and train models for a single task and evaluate on that task

Now: pretrain a model, and watch it do tasks it was not explicitly trained for

GPT-3: AVERAGE PERFORMANCE across 42 BENCHMARKS

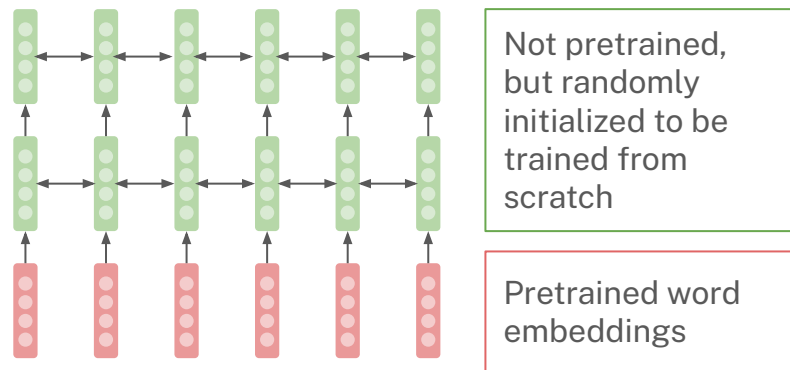
Source: OpenAI (Brown et al., 2020) | Chart: 2021 AI Index Report



Before

Circa 2017:

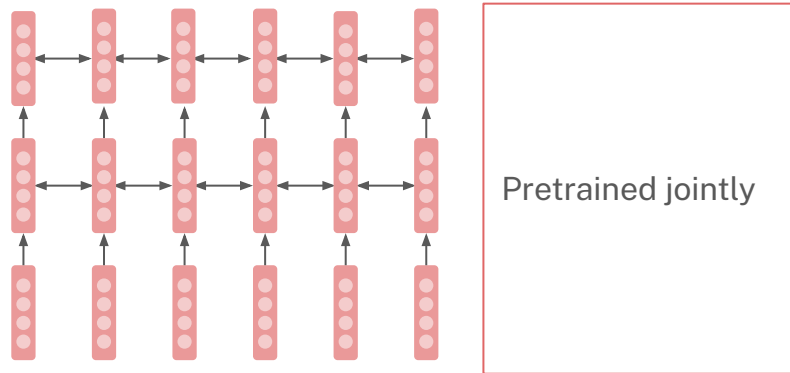
- Start with pretrained word embeddings
- Use an RNN or transformer to train on a task
- The training data for our task (e.g. question answering) must teach the model nearly everything it needs to know about language



Now

In modern NLP:

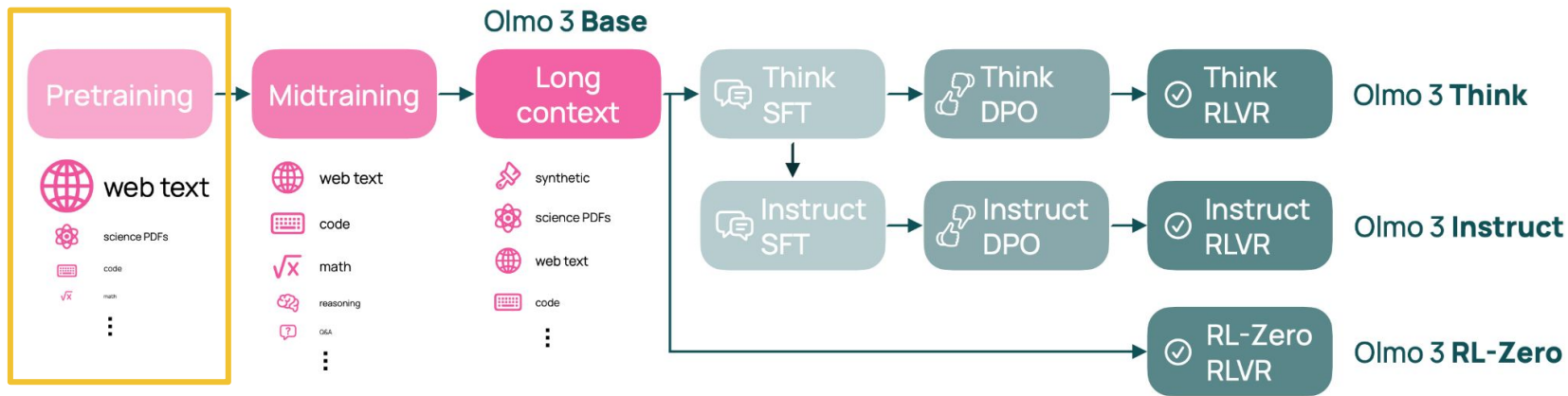
- All or almost all parameters are initialized by a pretrained base model
- Base models are *self-supervised*; they learn how to **reconstruct language data**
- This self-supervision builds strong representations and probability distributions from which we can sample and generate text



Why pre-train?

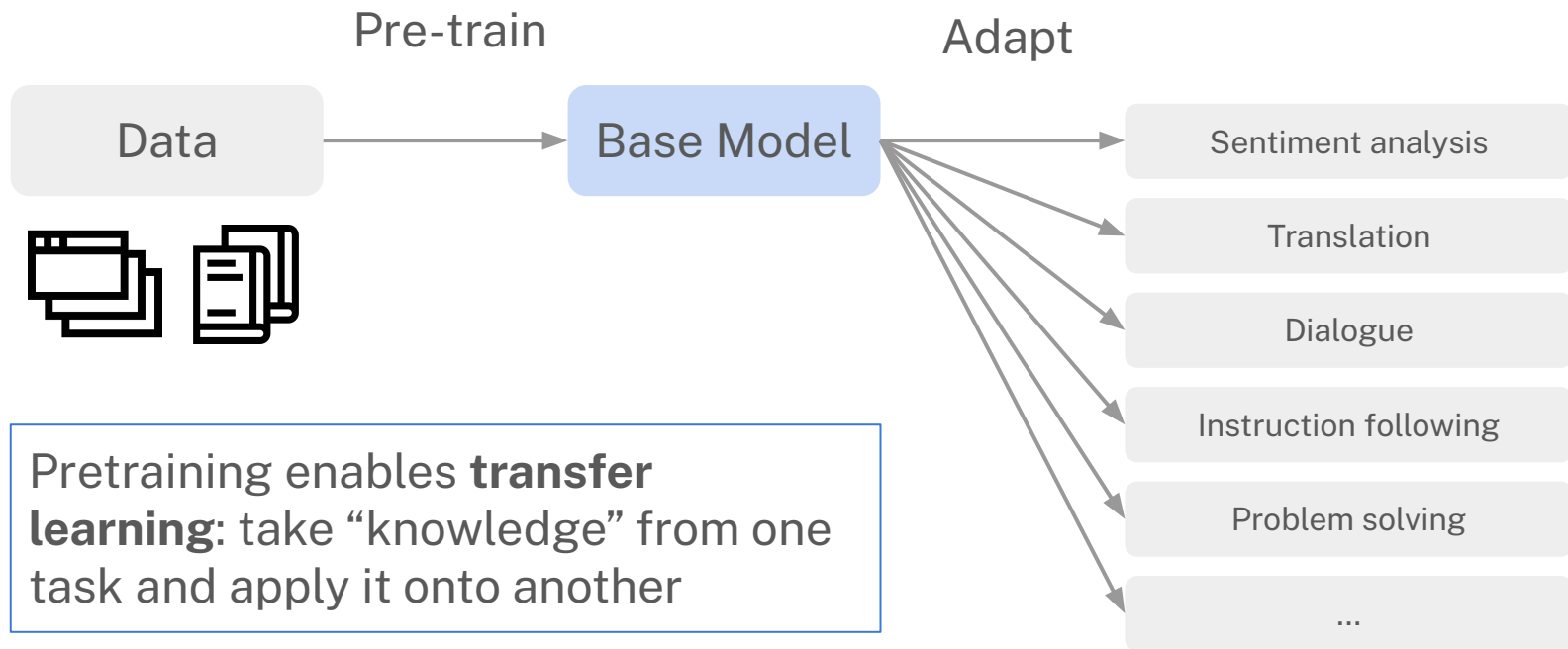
Most of what models “know” is learned during pretraining!

Most of a model’s training compute is spent at this stage, e.g. 90%, w/ much of it spent on experiments that inform the final pretraining run.



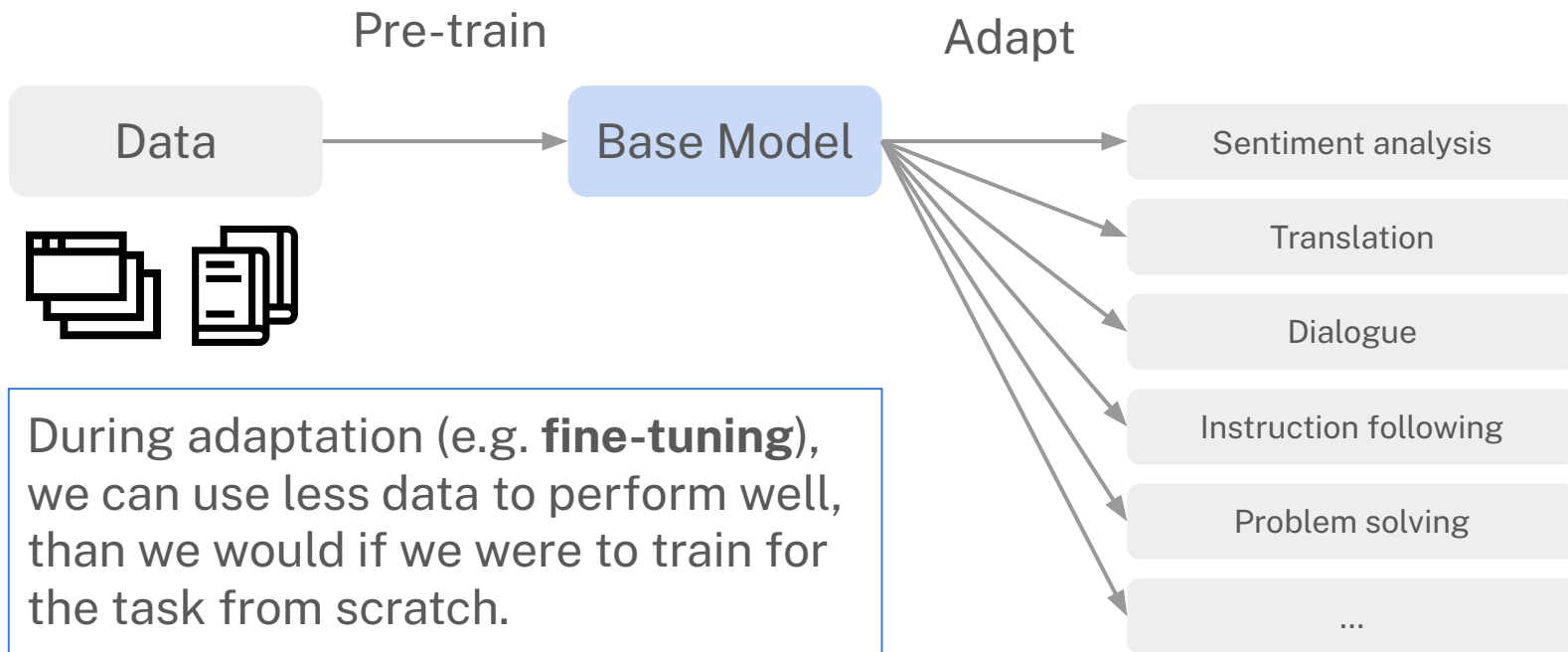
[Olmo 3 paper](#)

Basic idea



Basic idea

One model, many tasks!



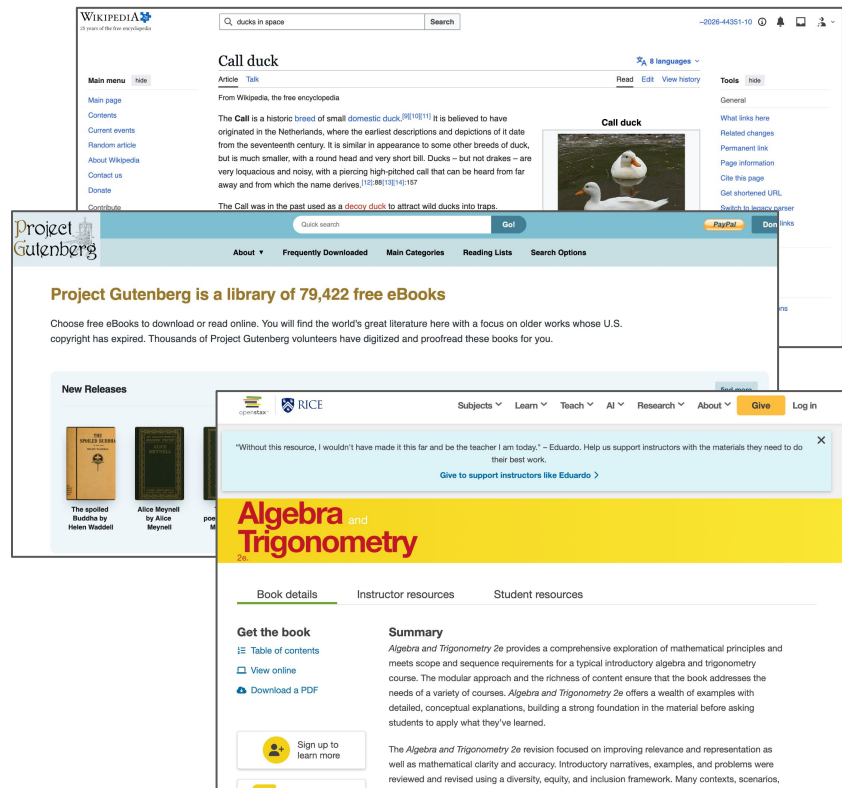
Pretraining is scalable

Self-supervised learning on large datasets allow pretraining **to scale**

Our pretraining objective cannot require human labeling effort

Our model architecture must be able to handle training on large, diverse datasets

The era of “big data”, e.g. Internet data, helped push this forward



Model Pretraining in Three Ways

Pretraining for three types of architectures

The neural architecture influences pretraining and natural use cases.

Architecture	Can be bidirectional?	Pretraining objective	Examples
Encoder	Yes	Masked language modeling	BERT, RoBERTa
Encoder-decoder	Yes, for the encoder part	Span corruption	T5, BART
Decoder	No	Causal / autoregressive language modeling	GPT-2, GPT-3, Llama

Pretraining for three types of architectures

The neural architecture influences pretraining and natural use cases.

Architecture	Can be bidirectional?	Pretraining objective	Examples
Encoder	Yes	Masked language modeling	BERT, RoBERTa
Encoder-decoder	Yes, for the encoder part	Span corruption	T5, BART
Decoder	No	Causal / autoregressive language modeling	GPT-2, GPT-3, Llama

BERT

Bidirectional Encoder Representations from Transformers

([Devlin et al. 2019](#))

Learn representations via pretraining, and further train (fine-tune) the model on some downstream task.

Bidirectional RNNs (LSTMs) +
next & prev word prediction!

Hmmmm



Pretrain on

- Masked language modeling
- Next sentence prediction, a.k.a. is this the actual next sentence or a random one
 - later found to be less helpful and unnecessary

Finetune on

- Specific tasks, including:
 - Sentence pair classification, e.g. entailment/contradiction
 - Named entity recognition
 - Passage-anchored question answering
 - Sentiment classification

BERT details

- Two models released:
 - **BERT-base**
 - 12 layers, 768 hidden size, 12 attention heads, 110M params
 - **BERT-large**
 - 24 layers, 1024 hidden size, 16 attention heads, 340M params
- **Pretraining data:** BooksCorpus (800M words) & English Wikipedia (2.5B words)
- **Compute:** Pretraining required 64 TPU chips for 4 days; finetuning possible on a single GPU in minutes to hours
- Produces on-demand **contextualized word embeddings**: *bank in I sat by the river bank produces a different internal representation than I robbed the bank.*

Masked language modeling

Since BERT uses **bidirectional context**, we can't do next word prediction!

Instead, we replace some fraction of words in the input with a special [MASK] token and predict those words. We call this **masked language modeling**.

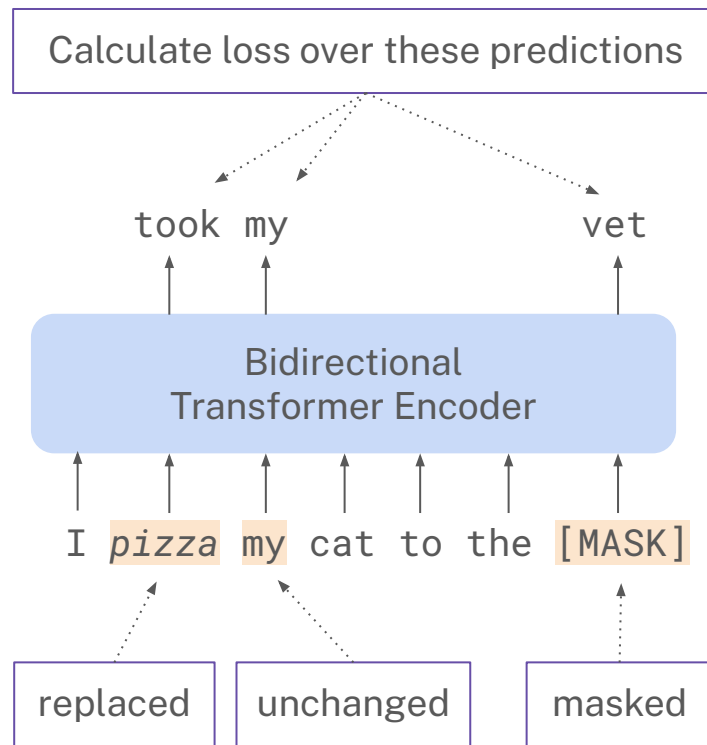
Let $\tilde{x}$ be the masked version of a sequence x . We're learning $p(x | \tilde{x})$:

After a TV [MASK] on her, I [MASK] my cat to the [MASK].

Masked language modeling

Masked language modeling in BERT

- Predict a random 15% of subword tokens. Only add loss terms for this subset, of which we:
 - 80%: replace with [MASK]
 - 10%: replace with a random token
 - 10%: leave token unchanged, but still predict it
- Not masking every word the model is asked to predict helps build strong representations of non-masked words
- No masks present during fine-tuning



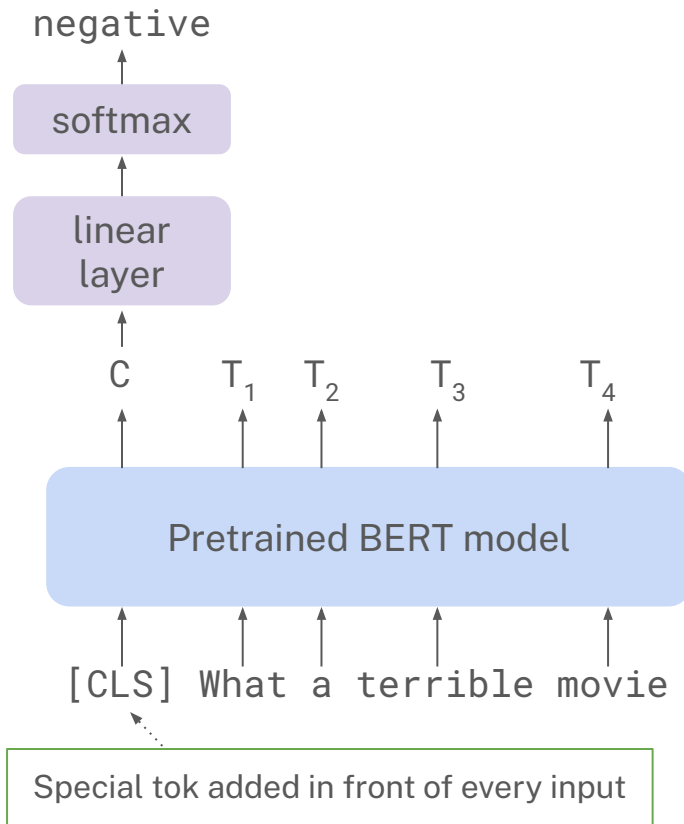
Adapting an MLM

Add an **output layer** that maps a hidden vector to scores

Finetune the weights (either just the linear layer or all the weights).

Classification example:

Given a dataset of movie reviews and positive, neutral, negative labels



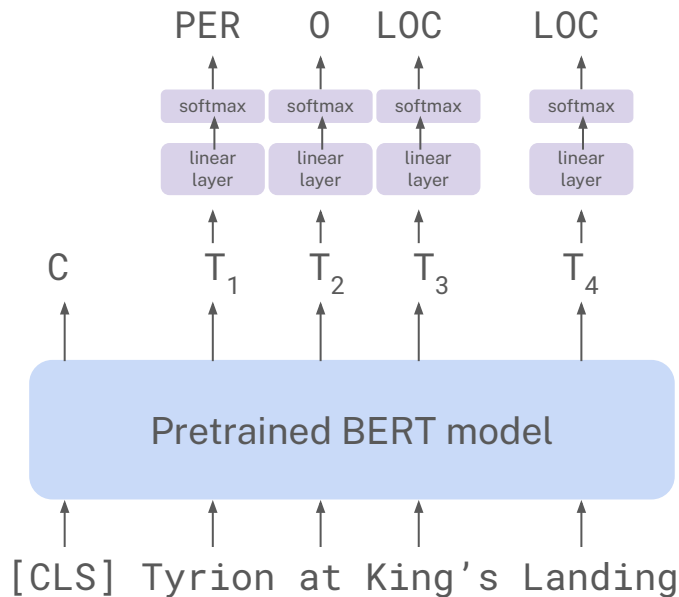
Adapting an MLM

Add an **output layer** that maps a hidden vector to scores

Finetune the weights (either just W or all the weights).

Sequence tagging example:

Given a dataset of book text, label all of the potential settings and characters



BERT

Bidirectional Encoder Representations from Transformers

([Devlin et al. 2019](#))

Pretrain once, finetune many times!



Highly successful: state-of-the-art performance on many tasks.

BERT became massively popular and versatile!

BERT variants

Some generally accepted improvements to the BERT pretraining formula:

- RoBERTa: train BERT for longer and remove next sentence prediction
 - More data + more compute + the same architecture = better results!

Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	94.6/89.4	90.2	96.4

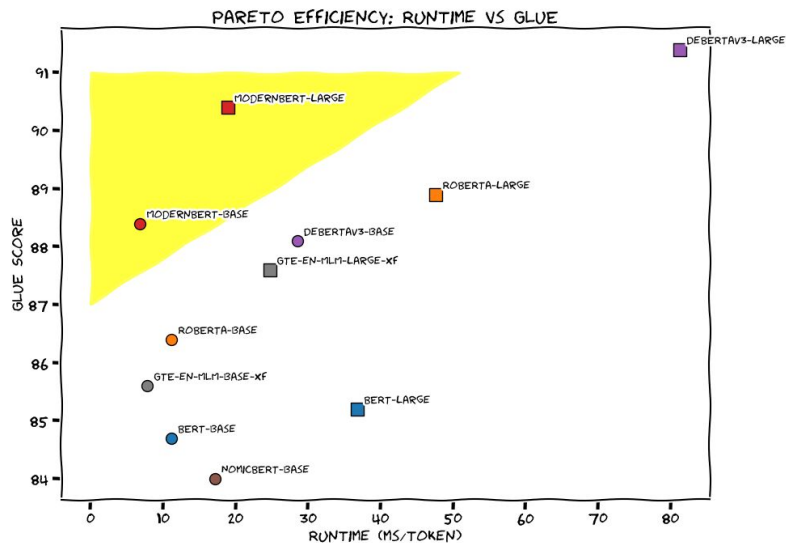
- SpanBERT: mask contiguous spans of tokens, e.g. irr## esi##, sti##, bly, makes a harder, more useful pretraining task

BERT variants

ModernBERT: “Upgrading a Honda Civic for the Race Track”

a.k.a. improving 2019 BERT with 2024 methods, including:

- Llama2-style (2023-style) transformer architecture, e.g. RoPE embeddings
- Alternating global and local attention instead of using only global attention
- New data scales and sources: 2 trillion tokens



[ModernBERT blog post](#)

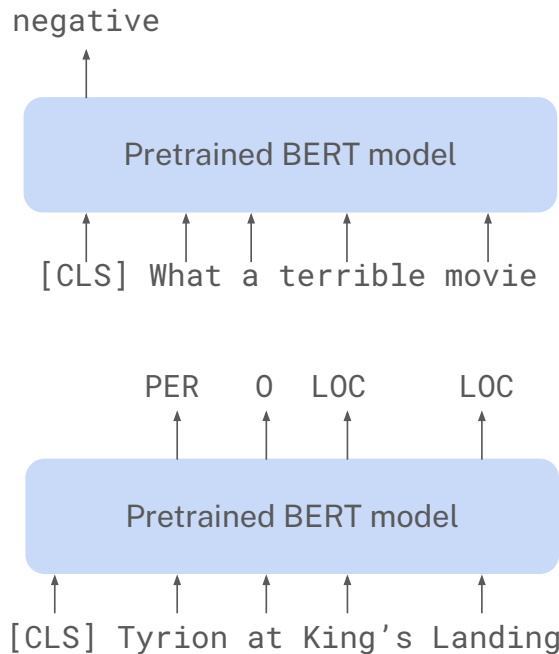
Limitations of pretrained encoders

Why not use pretrained encoders for everything?

BERT and other pretrained encoders don't naturally lead to nice autoregressive (one-word-at-a-time) text generation.

For example, free-form question answering!

If your task involves generating sequences, consider using a pretrained decoder!



Pretraining for three types of architectures

The neural architecture influences pretraining and natural use cases.

Architecture	Can be bidirectional?	Pretraining objective	Examples
Encoder	Yes	Masked language modeling	BERT, RoBERTa
Encoder-decoder	Yes, for the encoder part	Span corruption	T5, BART
Decoder	No	Causal / autoregressive language modeling	GPT-2, GPT-3, Llama

Pretraining encoder-decoders

Reminder:

- A prefix of every input is provided to the **encoder** and is not predicted.
- Outputs from the **decoder** are used to train the whole model.

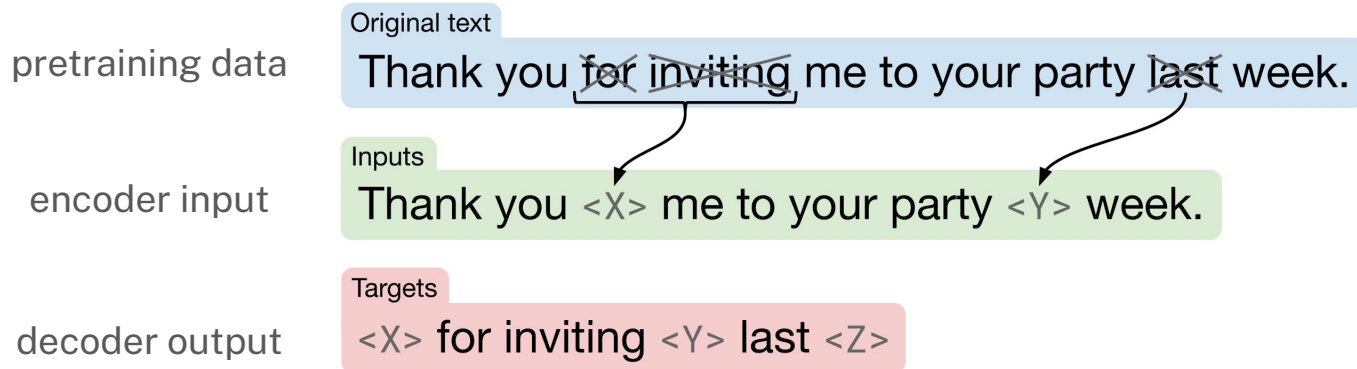
Example pretraining objective: **span corruption**

Used to pretrain the Text-to-Text Transfer Transformer (T5) by [Raffel et al. 2018](#)

T5's pretraining objective

Span corruption: replace different length spans from the input with unique placeholders, and then decode out the spans that were removed.

Inspired by BERT's masked language modeling!



Text-to-text transfer transformer (T5)

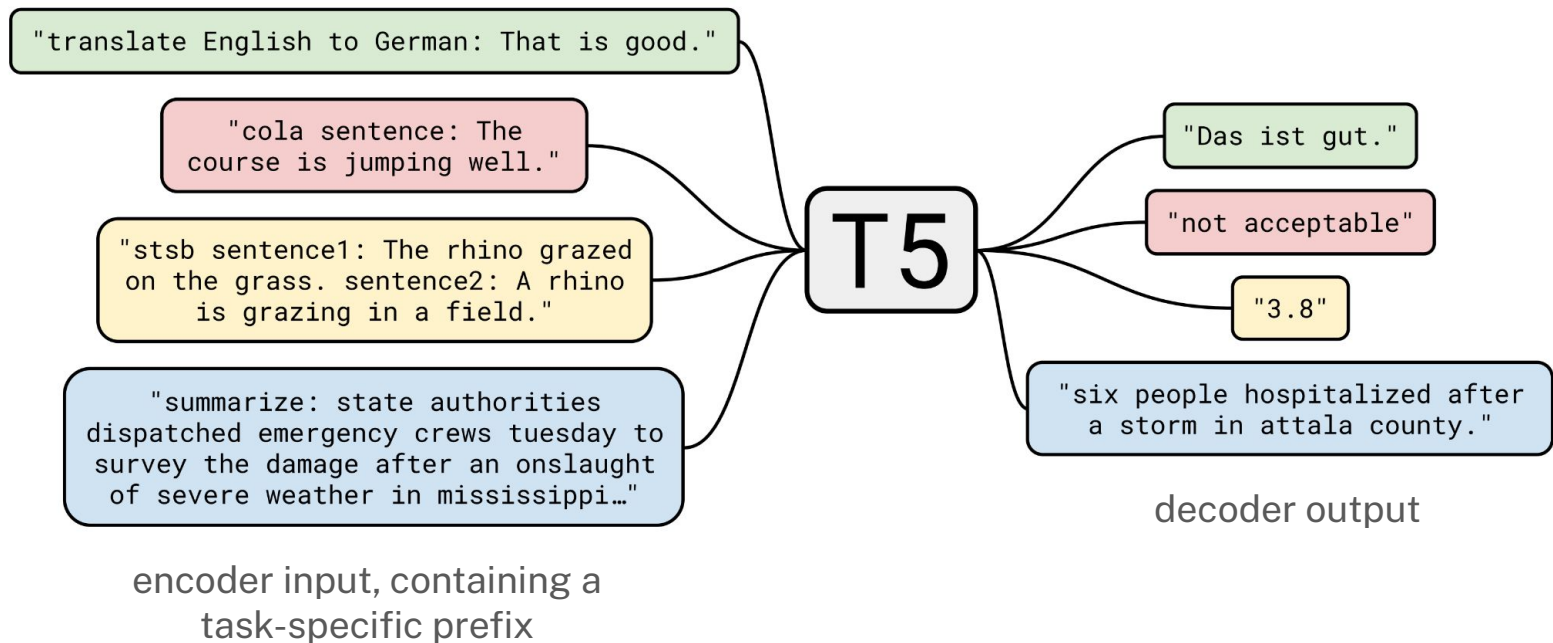
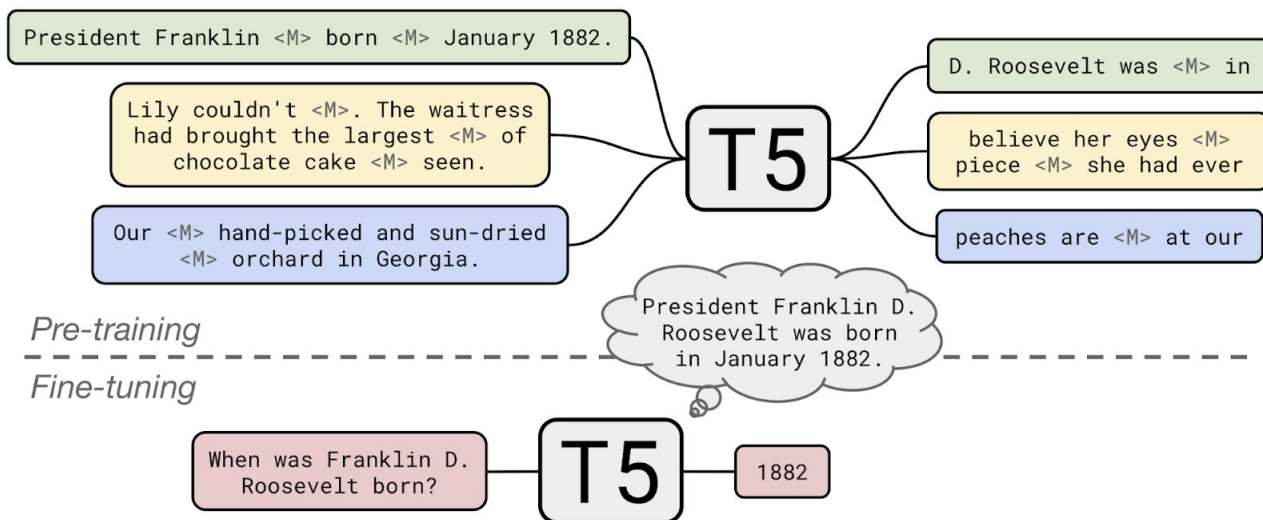


Figure from [Raffel et al. 2018](#)

Text-to-text transfer transformer (T5)

T5 can be finetuned to answer a wide range of questions, retrieving knowledge from its parameters that it learned during pre-training.



From [Google's T5 blog post](#)

Pretraining for three types of architectures

The neural architecture influences pretraining and natural use cases.

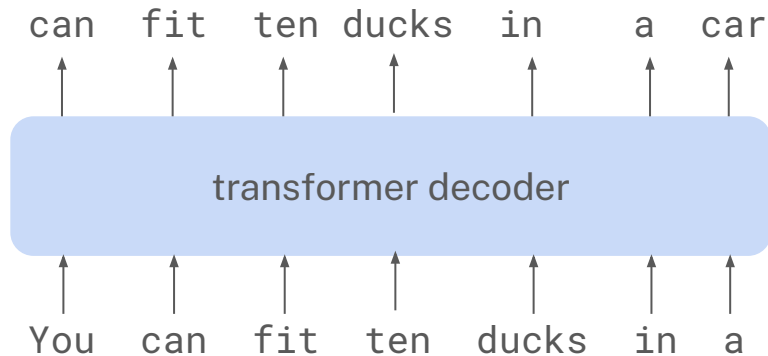
Architecture	Can be bidirectional?	Pretraining objective	Examples
Encoder	Yes	Masked language modeling	BERT, RoBERTa
Encoder-decoder	Yes, for the encoder part	Span corruption	T5, BART
Decoder	No	Causal / autoregressive language modeling	GPT-2, GPT-3, Llama

Pretraining decoders

Can pretrain decoders as language models and use them as text generators

Decoders predict the next token based on prior task-specific context, e.g.

- Dialogue: prior context is dialogue history
- Summarization: prior context is the document



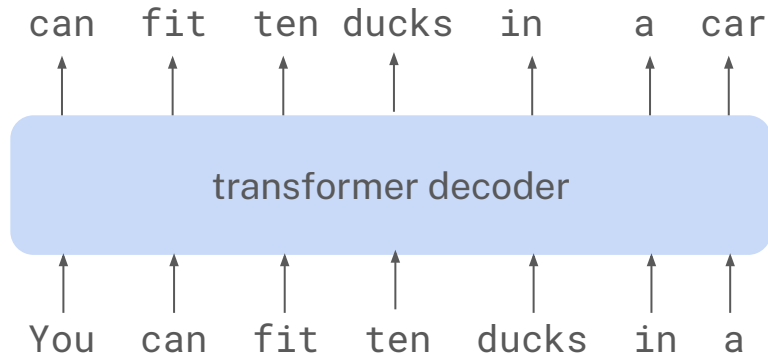
Pretraining decoders

The **Generative Pretrained Transformer (GPT)** was a big success in pretraining a unidirectional, left-to-right decoder

(Radford et al. 2018)

Trained on long spans of contiguous text (e.g. books), instead of sentences.

Training objective: **causal (autoregressive) language modeling**



Question

Which paper first used the term “GPT”?

- A) GPT paper from OpenAI
- B) GPT-2 paper from OpenAI
- C) GPT-3 paper from OpenAI
- D) BERT paper from Google

GPT-2: increasingly convincing generations

A larger (1.5B) version of GPT trained on more data

→ More delicious-sounding outputs than our RNN several lectures ago!

My 2 failures at Christmas morning were not due to poor frosting skills. That was another success story. The cake was beautifully moist, and well, just a beautiful cake. It's got very little air in it or moisture, but all the chocolate and mint was perfectly blended together.

One thing that made me cringe a bit was that all those ingredients – egg yolks, sugar, melted butter, chocolate chips, melted chocolate, and mint – had to be mixed right into the cake batter, which is super time-consuming.

GPT-3

GPT-3 had 175B parameters.

So far, we've interacted with pretrained models in two ways:

- Sample from the distributions they provide given some input or prompt
- Fine-tune them on a task we care about

Very large LMs, e.g. GPT-3, after pretraining, seemed to perform some kind of **learning *without gradient updates*** simply from examples you provide them!

GPT-3

In-context learning: use in-context examples to specify or *demonstrate* the task to be performed.

input

thanks -> merci

hello -> bonjour

mint -> menthe

otter ->

output

loutre

Prompt-based model adaptation!

From GPT to GPT-2 to GPT-3

Decoder-only transformer LMs and pretraining with a **language modeling objective** have emerged as the current winner, or modeling paradigm.

GPT
117M

Trained on ~1B tokens
June 2018

GPT-2
1.5B

Trained on ~14B tokens
Feb 2019

GPT-3
175B

Trained on ~300B tokens
May 2020

Why?

Bidirectionality in encoder-only models matters less at larger scales

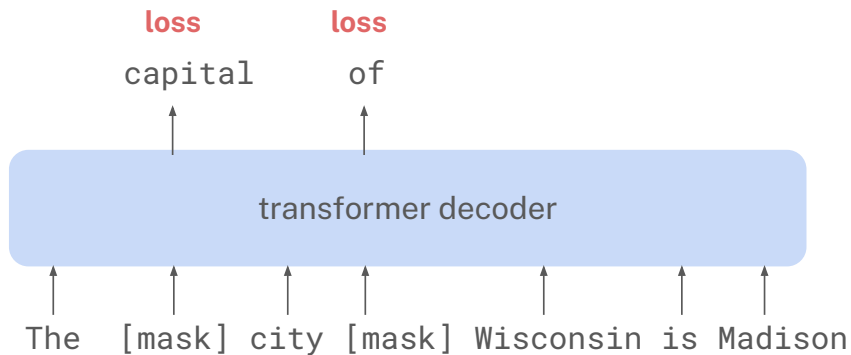
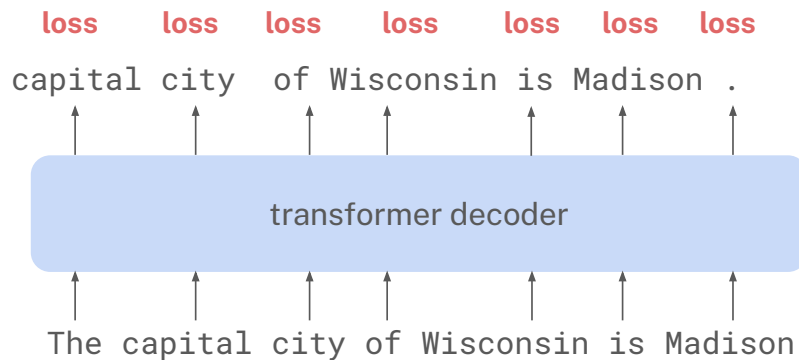
- Caveat: with fine-tuning, encoder-only masked LMs can sometimes perform better on specific tasks than decoder-only

Dropping the encoder from encoder-decoder didn't seem to hurt too much – and it saved on compute and memory

- Don't need separate token allocation budgets and padding for each component, and instead just have outputs directly follow input context

Why?

Denoising objectives (e.g. masking, span corruption) have less “loss exposure” and fewer forward-pass training signals than causal language modeling.



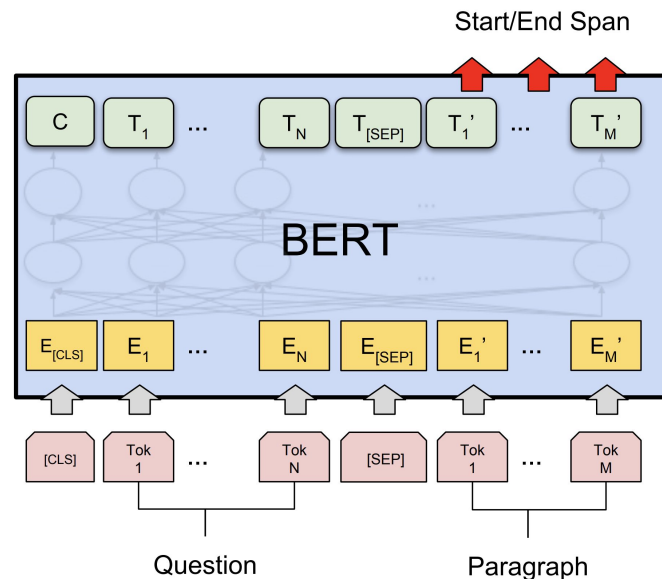
Why?

Masked LMs require too much consideration around how to format in-place, per-token predictions to work for different types of tasks.

Next-word prediction handles every task the same way.

Example: question answering

Causal LMs can be used for text generation, e.g. chats with users!

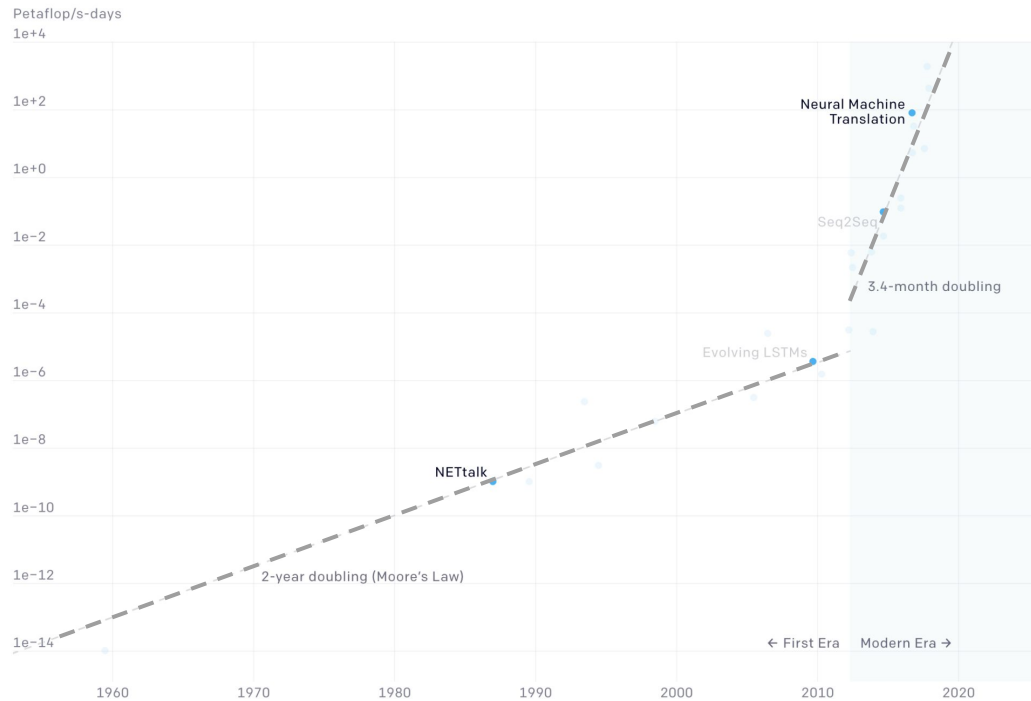


Question answering task format

Figure from [the BERT paper](#)

Scaling Pretraining

We are using exponentially more compute over time



From [OpenAI](#)

Scaling laws motivation

Even the GPU-rich don't have infinite compute (\$\$) to train an LM.

Training GPT-3 costs millions of dollars.

Failed yolo runs are expensive!

Question: given some budget of *compute*

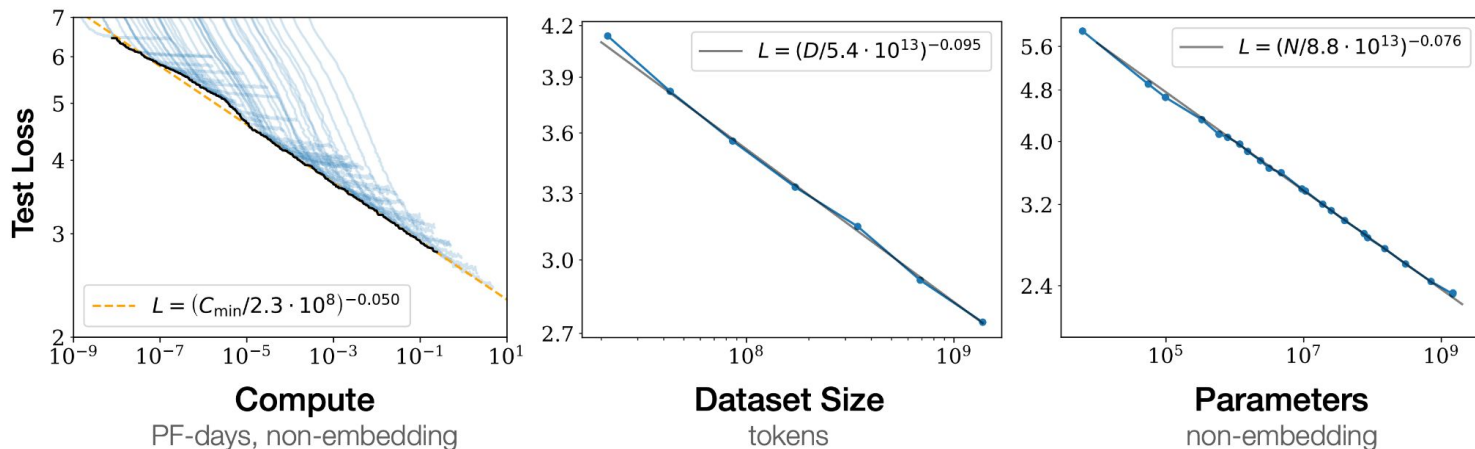
And perhaps some *target performance*,

What model (e.g. *size, hyperparameters*) should I train

On how much *data*?

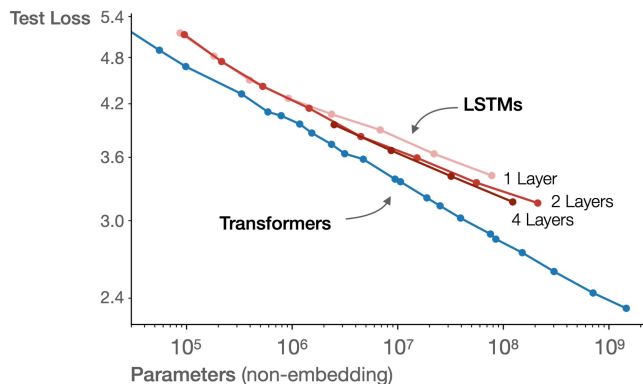
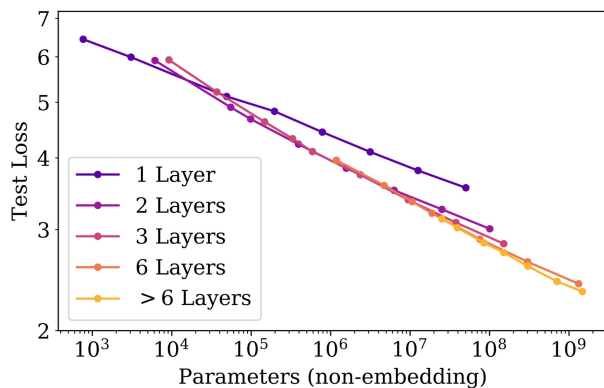
Answer: let's predict bigger runs from smaller ones!

How does the test loss scale with ...



Empirically, scaling up models leads to predictable gains in perplexity, with a **power law** relationship ([Kaplan et al. 2020](#))

How does the test loss scale with ...



Empirically, scaling up models leads to predictable gains in perplexity, with a **power law** relationship ([Kaplan et al. 2020](#))

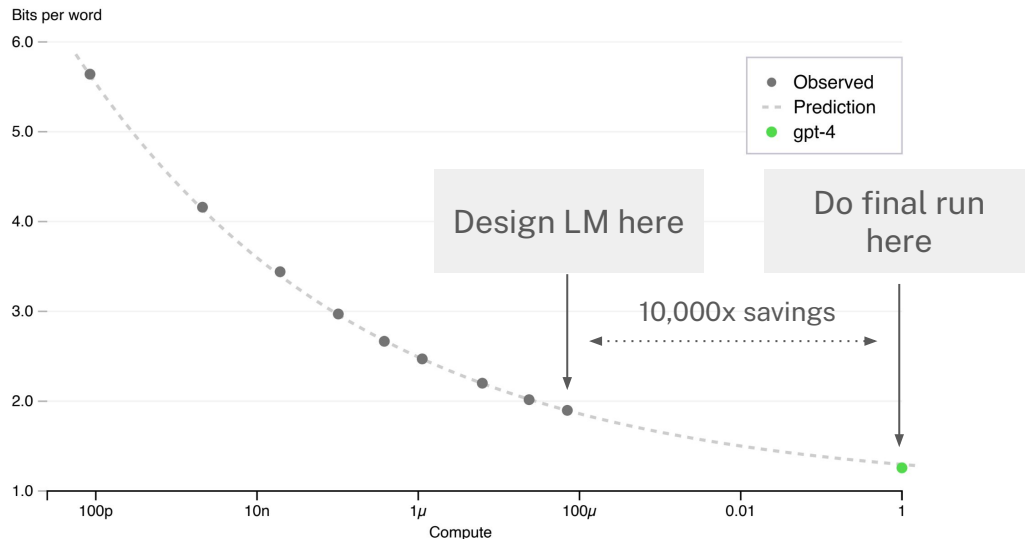
Example: pretraining GPT-4

With a limited budget, you cannot do a hyperparameter sweep

Instead, run many experiments at a small scale

Use a scaling law to figure out the best hyperparam setup *and* predict your final performance on a large-scale pretraining run

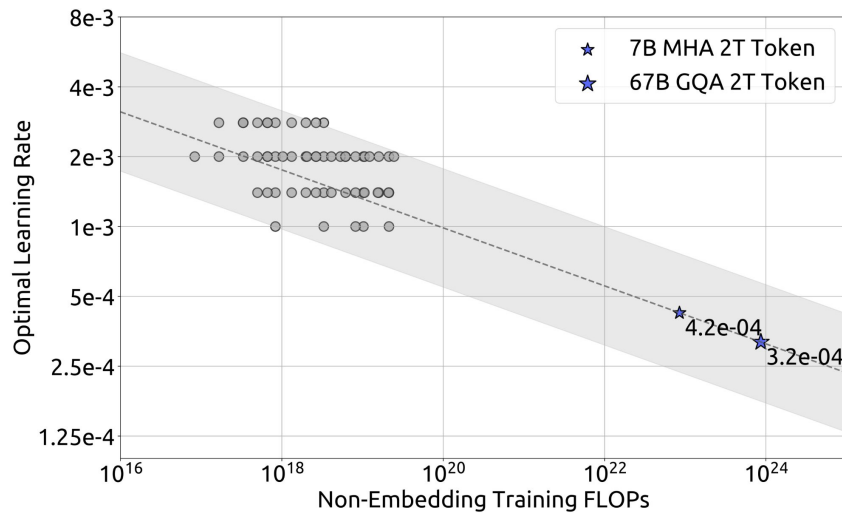
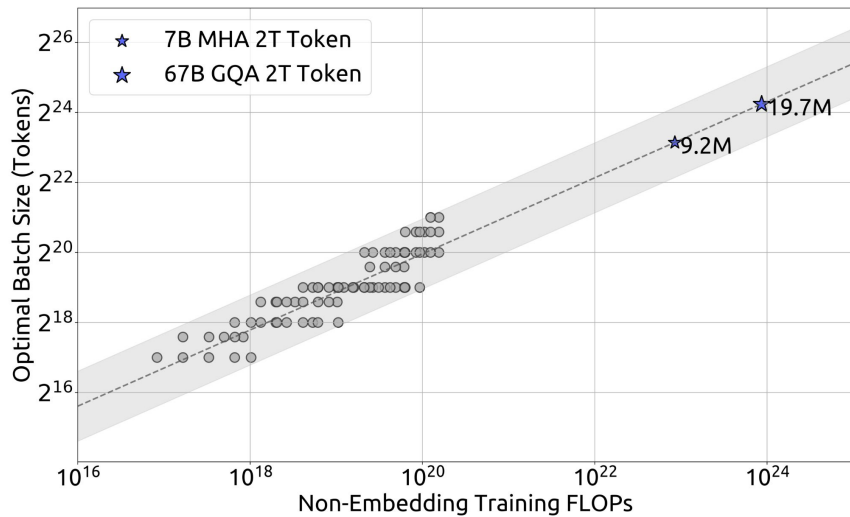
OpenAI codebase next word prediction



From [GPT-4 technical report](#)

Example: Deepseek

Through extensive experiments at smaller scale, modeled the power law between compute budget and the “best” batch size and learning rate



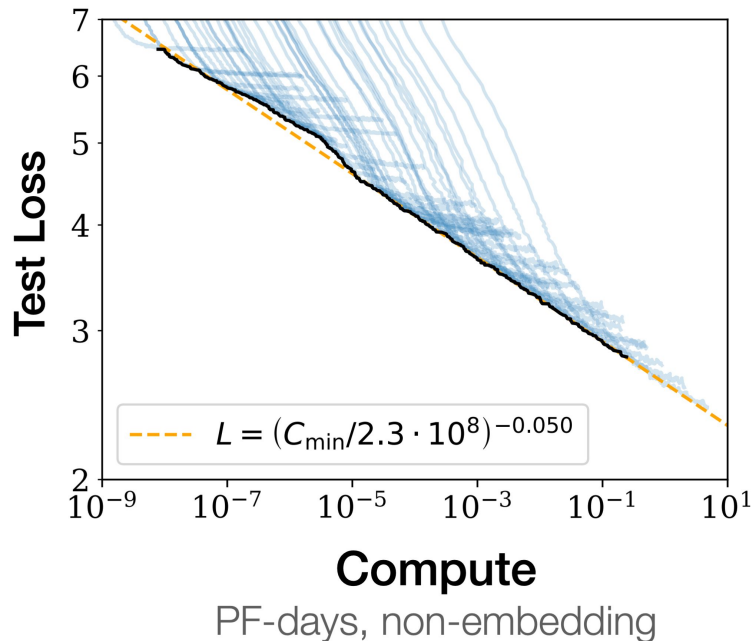
From [Deepseek LLM paper](#)

Kaplan scaling laws

Kaplan et al. 2020 trained autoregressive transformers, varying:

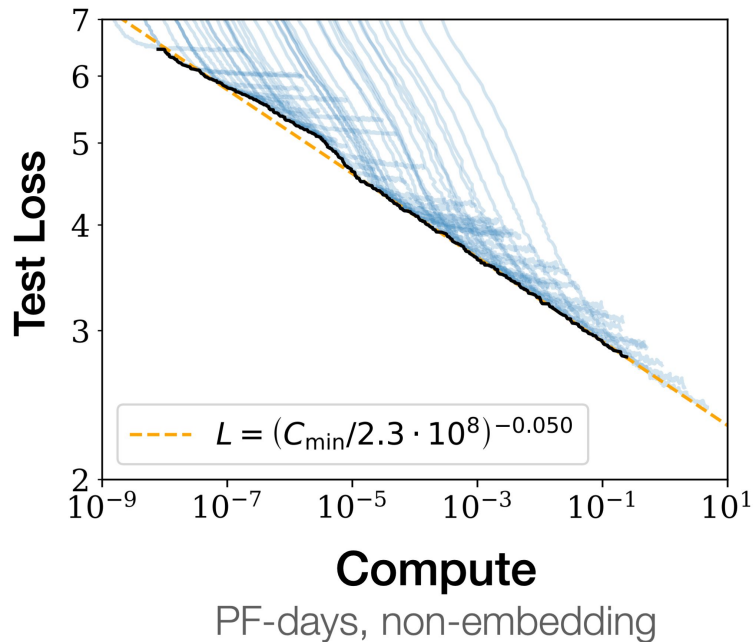
- Model size: 768M - 1.5B
- Data (one epoch): 22M - 23B tokens
- Shape e.g. depth, attention heads, width
- Batch size

Compute estimate = 6 x num params x
batch size x num steps



Kaplan scaling laws

1. Plot loss at each step of training (light blue)
2. Pick minimum loss at each amount of compute (black, a.k.a. “compute optimal”)
3. Run linear regression on the resulting loss vs. compute plot (orange dashed line, a.k.a. “scaling law”)

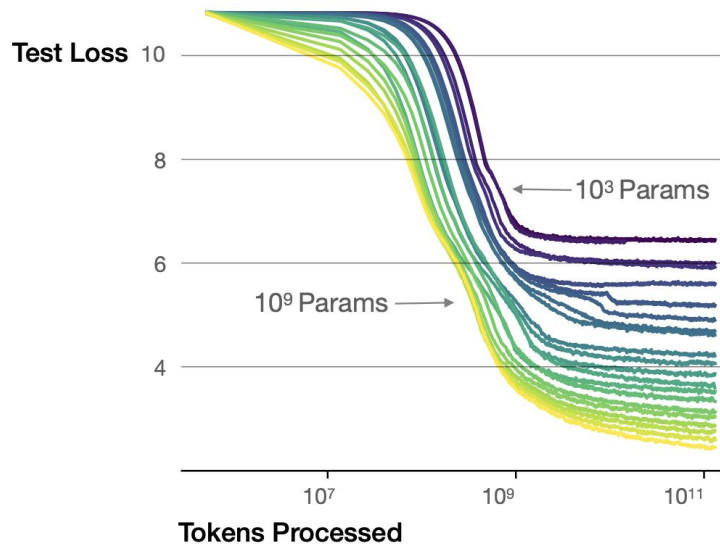


Kaplan scaling laws' findings

Large models are more sample-efficient, require less data, than small models.

Performance depends *strongly* on number of parameters, dataset size, and amount of compute used.

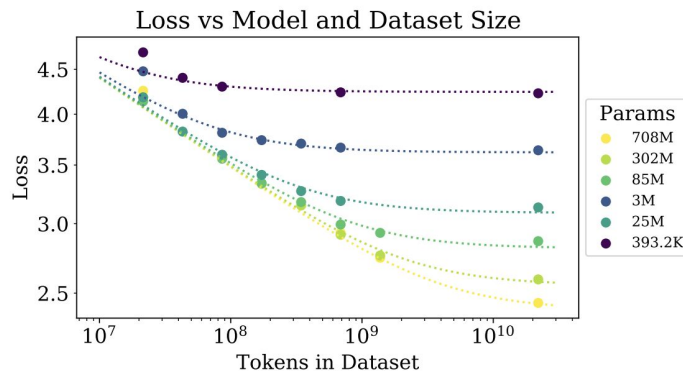
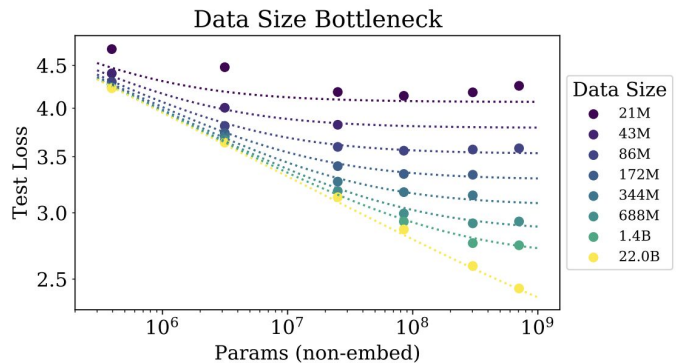
It depends more *weakly* on model shape e.g. depth (number of stacked layers), width (hidden size of layers).



Kaplan scaling laws' findings

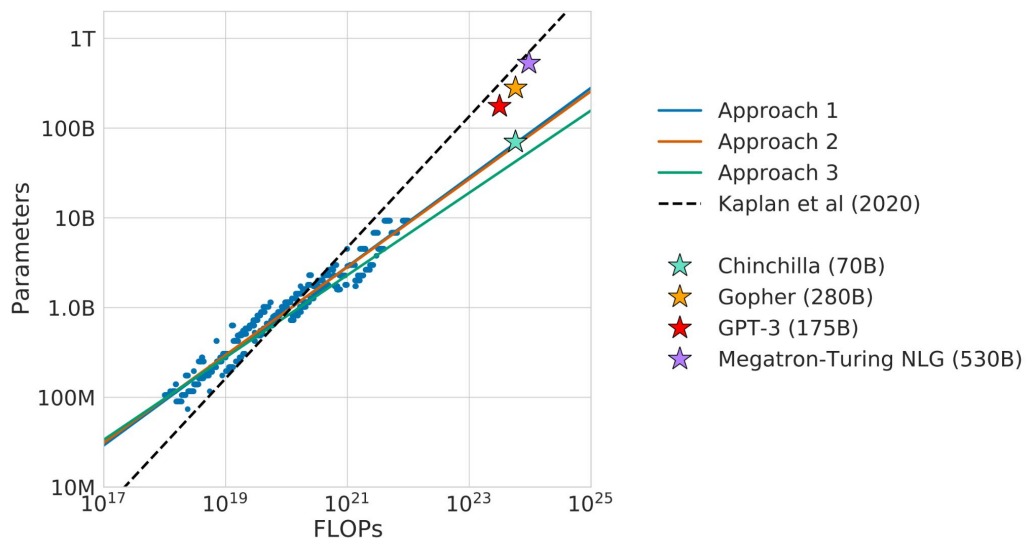
Number of parameters and dataset size should be scaled up in tandem, so that they don't bottleneck each other.

Every time we increase the model size 8x, increase the data by roughly 5x to avoid an overfitting penalty.



Chinchilla's scaling laws

The Chinchilla paper argues that Kaplan's scaling laws are a bit off



[Hoffmann et al. 2022](#)

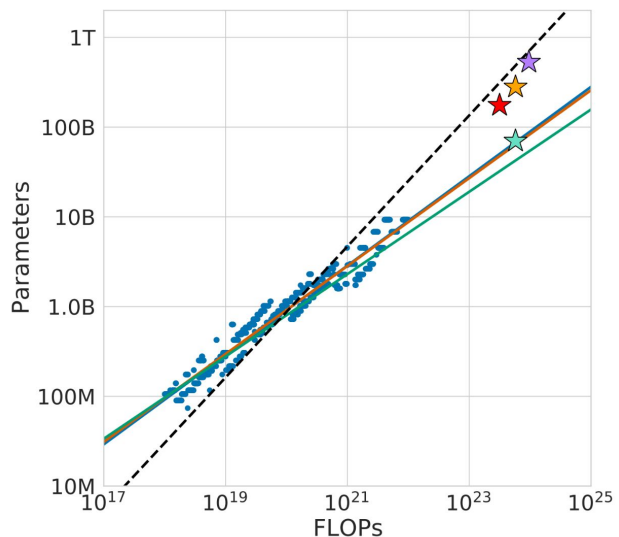
Approach 1: Fix model sizes and vary number of training tokens

Approach 2: vary the model size for a fixed set of nine different training FLOP counts

Approach 3: Model all final losses from experiments in Approach 1 & 2 as a function of model param count and number of tokens

Chinchilla's scaling laws

The Chinchilla paper argues that Kaplan's scaling laws are a bit off



[Hoffmann et al. 2022](#)

- Approach 1
- Approach 2
- Approach 3
- Kaplan et al (2020)

- Chinchilla (70B)
- Gopher (280B)
- GPT-3 (175B)
- Megatron-Turing NLG (530B)

Grow the model faster than you grow your data

$$N_{\text{opt}} \propto C^{0.73}$$

For every doubling of model size, also double your data

$$N_{\text{opt}} \propto C^{0.5}$$

Reasons for the diff: Kaplan experimented w/ smaller models, and did not count embedding parameters

Also see: [Pearce & Song 2024](#), [Porian et al. 2024](#)

Scaling efficiency

GPT-3 didn't strike the right parameter-token sweet spot, but Chinchilla does!

This allows Chinchilla, a 70B model, to perform better than other larger LMs

Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
<i>Gopher</i> (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
<i>Chinchilla</i>	70 Billion	1.4 Trillion

MMLU performance	
Random	25.0%
Average human rater	34.5%
GPT-3 5-shot	43.9%
<i>Gopher</i> 5-shot	60.0%
<i>Chinchilla</i> 5-shot	67.6%
Average human expert performance	89.8%

[Hoffmann et al. 2022](#)

Train-optimal might not actually be what you want

Chinchilla tells you, given some **training** compute budget, what's the optimal data and model size combo to train.

However, **most of the compute in real deployment is inference.**

So, instead, industry developers favor training smaller LMs on more data than what the compute-optimal ratio suggests.

→ This smaller LM w/ more data can match the smarts of the larger LM, yet be cheaper to serve for users.

The more usage you expect, the more it becomes worth to pay an upfront cost.

Recap: scaling law recipe

1. Grid search w/ different parameters, varying two key scaling dimensions: model size (parameter count) and training length (data size)
2. Identify the Pareto frontier: given the fixed training budget, what is the best model config?
3. Fit simple regressions to the results to make trends predictable beyond the tested scale
4. Show it extrapolates!
5. Profit (unless you're an open model developer, I guess ...)

Scaling laws

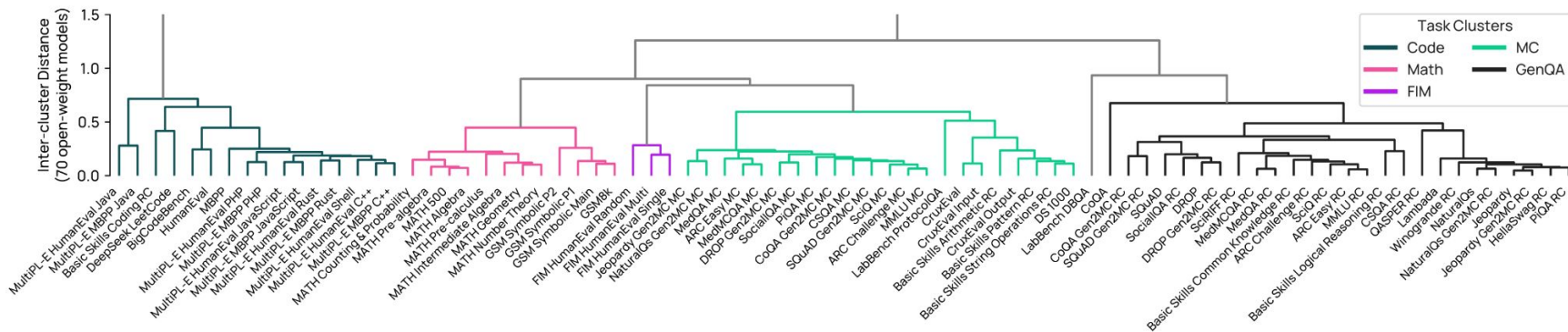
... are not laws! They're empirical trends

- They exist out of necessity: it's either this or yolo runs and inefficient resource allocation.
- Some argue that performance on some tasks may not scale predictably, and pretraining loss might not always relate directly to specific tasks ([Lourie et al. 2025](#))
- Mapping out scaling laws is expensive, and frontier labs rely heavily on internal secret scaling laws.

How to choose evaluation?

During pretraining runs, you can evaluate on perplexity (loss), but also helpful to see how the model is doing on concrete tasks.

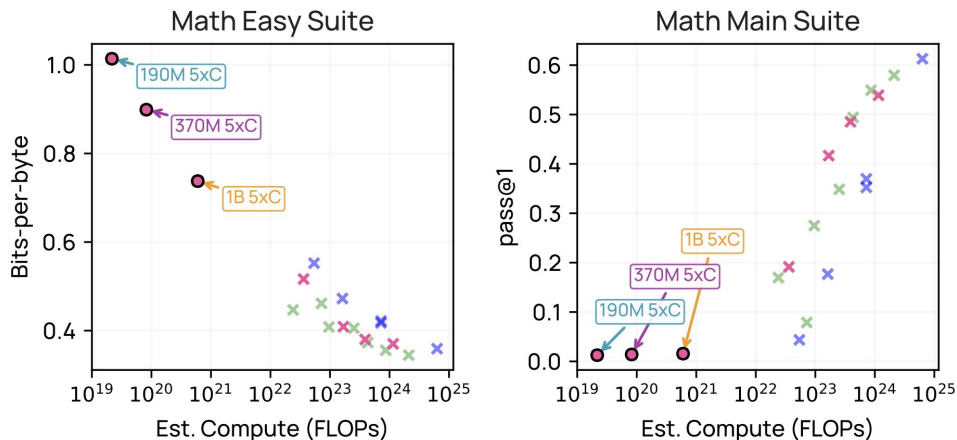
In the [Olmo 3 paper](#), they cluster evaluation tasks into categories, based on similarities in how they rank models:



How to choose evaluation?

Olmo 3 developers then determine the compute scale at which different tasks and metrics actually give useful performance discriminability.

Some eval is hard/noisier at smaller scales, and some saturate at higher scales.



Mid-training

Two purposes:

- **Data annealing:** integrate high-quality domain knowledge
 - Typically: math problems, coding challenges, and science reasoning datasets
- **Context length extension:** integrate longer inputs
 - Training w/ long sequences is costly, so pretraining is done on shorter sequences and longer input docs are saved for this later stage

Mid-training efforts are most effective when applied *after* pretraining, rather than at earlier stages, and can amplify the effects of later post-training

Post-mid pre-post training

Just kidding this doesn't exist (yet)

Pretraining via experience-backed intuition

“An incredible skill that I have witnessed, especially at OpenAI, is the ability to make ‘yolo runs’ work ...

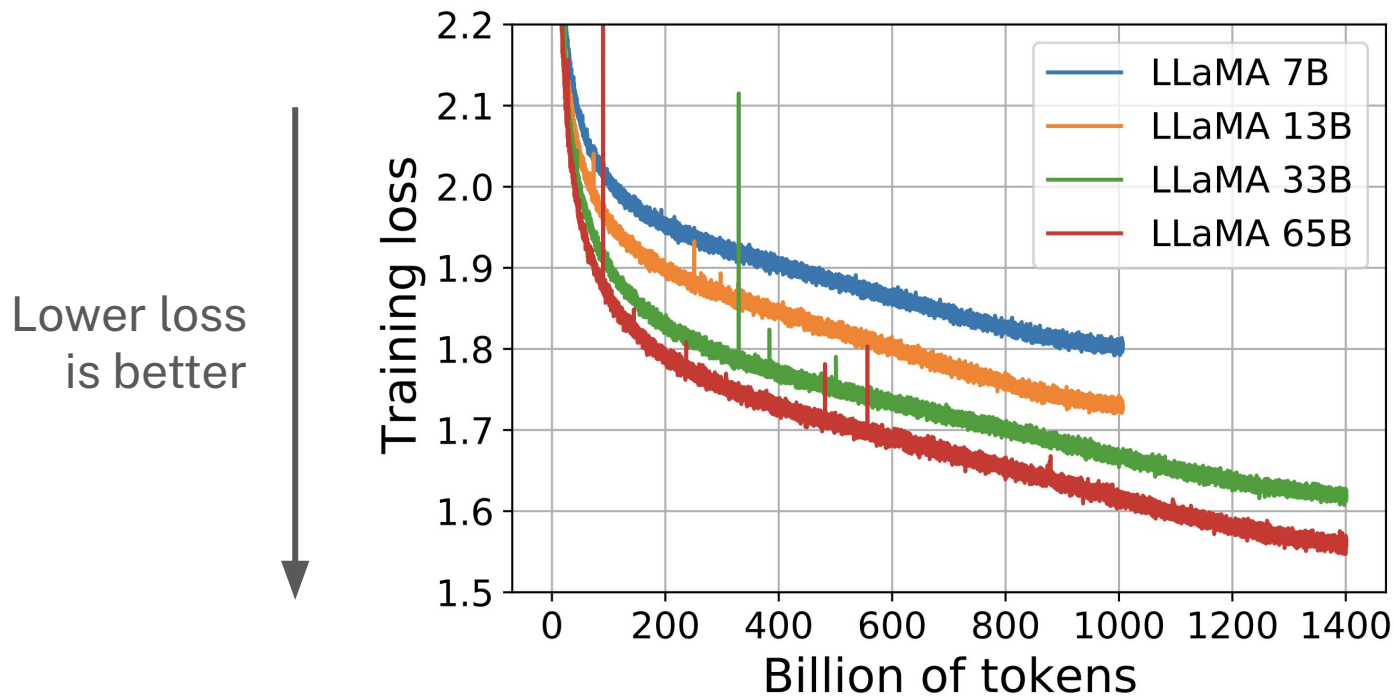
The researcher doing the yolo run relies primarily on intuition to set hyperparameter values, decide what parts of the model matter, and anticipate potential problems. These choices are non-obvious to everyone else on the team ...

I do not know how these researchers do it; my best guess is intuition built up from decades of running experiments, a deep understanding of what matters to make a language model successful, and maybe a little bit of divine benevolence. But what I do know is that the people who can do this are surely 10-100x AI researchers. They should be given as many GPUs as they want and be protected like unicorns.”

- Jason Wei on [Twitter/X](#), 2024

Pretraining Data

Pretraining data



From [the LLaMA paper](#)

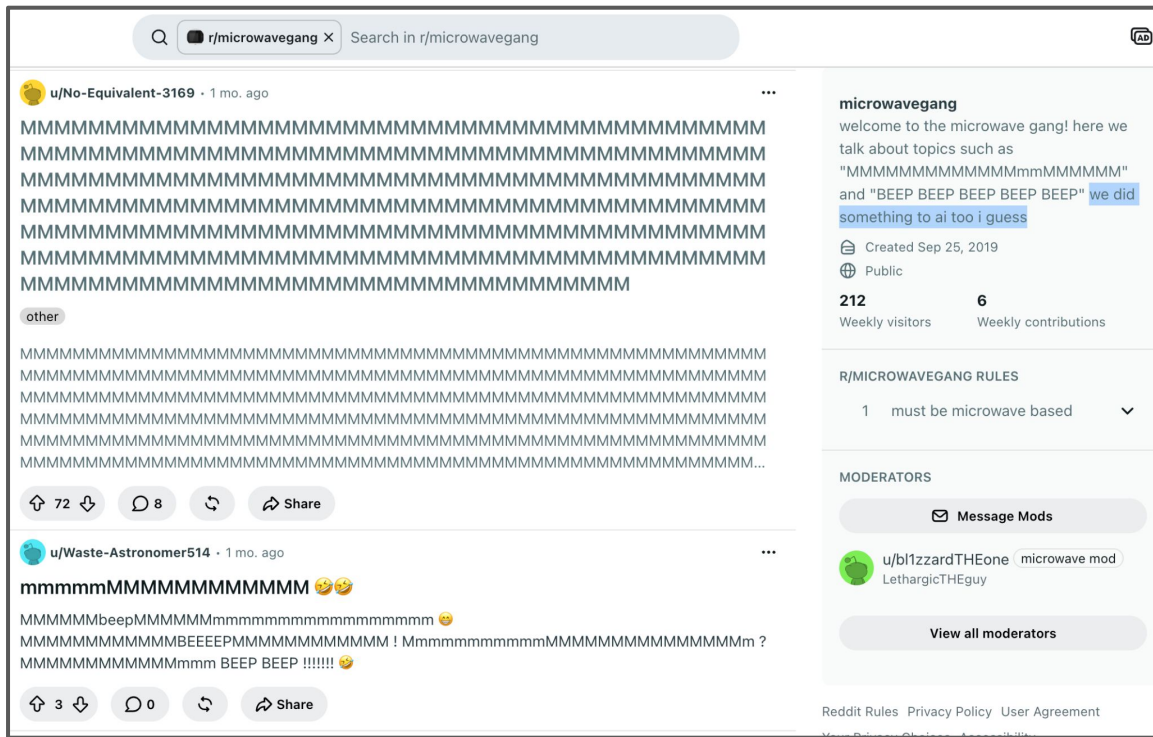
Data factors

Quantity: How much do I have?

Quality: Is it beneficial for my model?

Coverage: Does the data cover the domain(s) I care about, and in the right proportions?

Data factors



Where does this data come from?

BERT

Pre-training data The pre-training procedure largely follows the existing literature on language model pre-training. For the pre-training corpus we use the BooksCorpus (800M words) (Zhu et al., 2015) and English Wikipedia (2,500M words). For Wikipedia we extract only the text passages and ignore lists, tables, and headers. It is criti-

Where does this data come from?

BERT

Pre-training data The pre-training procedure

largely follows the model pre-training.

use the BooksCorpus (2015) and English

For Wikipedia we and ignore lists, ta

GPT

Instead, we created a new web scrape which emphasizes document quality. To do this we only scraped web pages which have been curated/filtered by humans. Manually filtering a full web scrape would be exceptionally expensive so as a starting point, we scraped all outbound links from Reddit, a social media platform, which received at least 3 karma. This can be thought of as a heuristic indicator for whether other users found the link interesting, educational, or just funny.

Where does this data come from?

BERT

Pre-training data

largely follows the model pre-training. use the BooksCorp (2015) and English For Wikipedia we and ignore lists, ta

GPT

Instead, we created a new web scrape which emphasizes document quality. To do this we only scraped web pages which have been curated/filtered by humans. Manually filtering a full web so as a starting p Reddit, a social karma. This can whether other us or just funny.

GPT-3

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Where does this data come from?

BERT

GPT-4

This report focuses on the capabilities, limitations, and safety properties of GPT-4. GPT-4 is a Transformer-style model [39] pre-trained to predict the next token in a document, using both publicly available data (such as internet data) and data licensed from third-party providers. The model was then fine-tuned using Reinforcement Learning from Human Feedback (RLHF) [40]. Given both the competitive landscape and the safety implications of large-scale models like GPT-4, this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar.

Dataset	(tokens)	training mix	training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Where does this data come from?

BERT

Pre-tr
largely
mod
use
201
For
and

This report focuses on the capabilities, limitations, and safety properties of GPT-4. GPT-4 is a

Llama 2

2.1 Pretraining Data

Our training corpus includes a new mix of data from publicly available sources, which does not include data from Meta’s products or services. We made an effort to remove data from certain sites known to contain a high volume of personal information about private individuals. We trained on 2 trillion tokens of data as this provides a good performance–cost trade-off, up-sampling the most factual sources in an effort to increase knowledge and dampen hallucinations.

or just funny.

WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Think pair share

Each section of the classroom will be assigned an **open** pretraining dataset, listed on the next page.

In three minutes: figure out and discuss w/ your neighbors:

- 1) Who released this dataset?
- 2) Where does the data for this dataset come from?
- 3) What are 2-3 data preprocessing decisions the creators of the dataset made?
- 4) What model, if any, was trained using this data in the dataset paper?

Nominate one person in the group to represent and share if called upon.

Open pretraining datasets

Dataset Name	Example LMs	Tokens	Read about it here
C4 (Oct 2019)	T5, FLAN-T5	175B	Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (Raffel et al. 2020) Documenting Large Webtext Corpora: A Case Study on the Colossal Clean Crawled Corpus (Dodge et al. 2021)
The Pile (Dec 2020)	GPT-J, Pythia, GPT-NeoX	387B	The Pile: An 800GB Dataset of Diverse Text for Language Modeling (Gao et al. 2020)
Dolma v1 (Aug 2023)	OLMo	3.1T	Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research (Soldaini et al. 2024)
DCLM (June 2024)	DCLM-Baseline	4T	DataComp-LM: In search of the next generation of training sets for language models (Li et al. 2024)
Common Corpus (June 2025)	PleIAs	2T	Common Corpus: The Largest Collection of Ethical Data for LLM Pre-Training (Langlais et al. 2026)

Why I care about pretraining

: an Open Corpus of Three Trillion Tokens
for Language Model Pretraining Research

Luca Soldaini ^{♥_α} Rodney Kinney ^{♥_α} Akshita Bhagia ^{♥_α} Dustin Schwenk ^{♥_α}

David Atkinson ^α Russell Author ^α Ben Bogin ^{α_ω} Khyathi Chandu ^α

Jennifer Dumas ^α Yanai Elazar ^{α_ω} Valentin Hofmann ^α Ananya Harsh Jha ^α

Sachin Kumar ^α Li Lucy ^β Xinxin Lyu ^ω Nathan Lambert ^α Ian Magnusson ^α

Jacob Morrison ^α Niklas Muennighoff ^α Aakanksha Naik ^α Crystal Nam ^α

Matthew E. Peters ^σ Abhilasha Ravichander ^α Kyle Richardson ^α Zejiang Shen ^τ

Emma Strubell ^{χ_α} Nishant Subramani ^{χ_α} Oyvind Tafjord ^α Pete Walsh ^α

Luke Zettlemoyer ^ω Noah A. Smith ^{α_ω} Hannaneh Hajishirzi ^{α_ω}

Iz Beltagy ^α Dirk Groeneveld ^α Jesse Dodge ^α

Kyle Lo ^{♥_α}

^αAllen Institute for AI ^βUniversity of California, Berkeley ^χCarnegie Mellon University

^σSpiffy AI ^τMassachusetts Institute of Technology ^ωUniversity of Washington

{lucas,kylel}@allenai.org

Why I care about pretraining

doLM: an Open Corpus of Three
for Language Model Pretraining

Luca S.   

Rodney Kinney  

Akshita Bha

David Atkinson  Russell Authur  Ben Bogin 

Jennifer Dumas  Yanai Elazar   Valentin Hofma

Sachin Kumar  Li Lucy  Xinxu Lyu  Nathan La

Jacob Morrison  Niklas Muennighoff  Aakanksh

yle I

rvin

Han

Jes

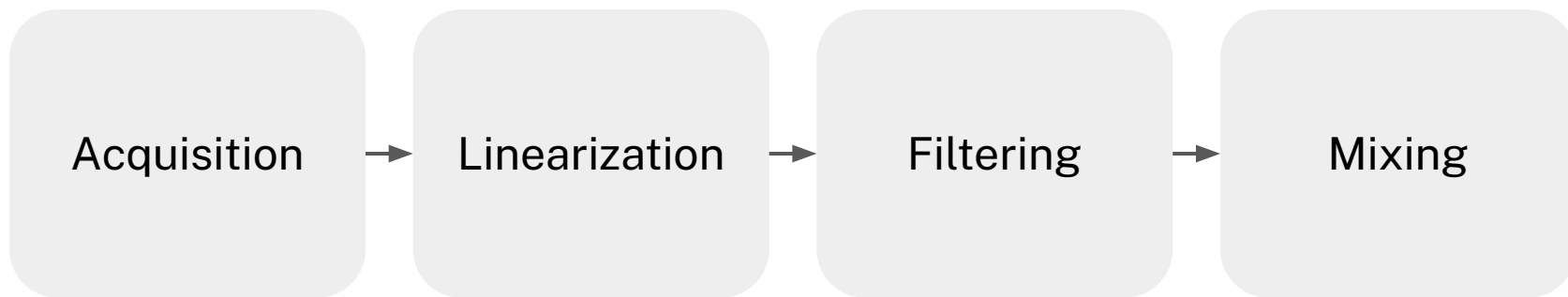
kele

gy

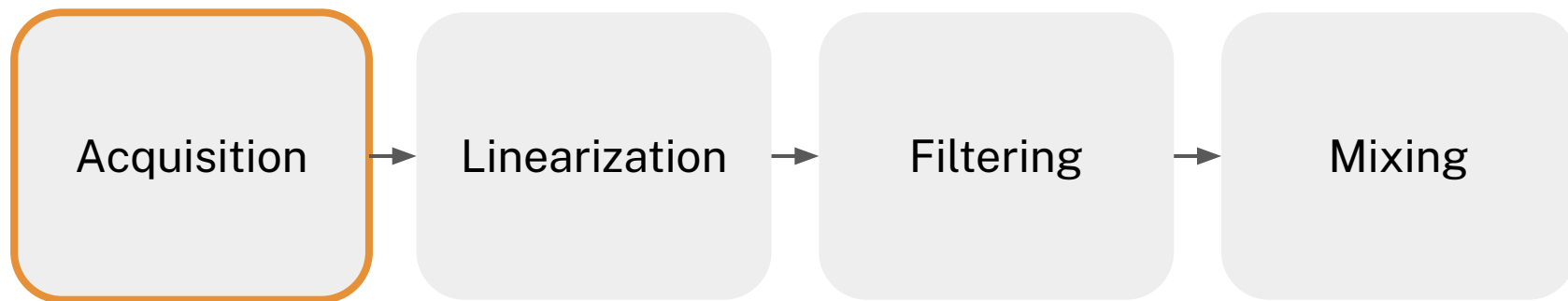


{lucas,kylel}@allenai.org

Data curation



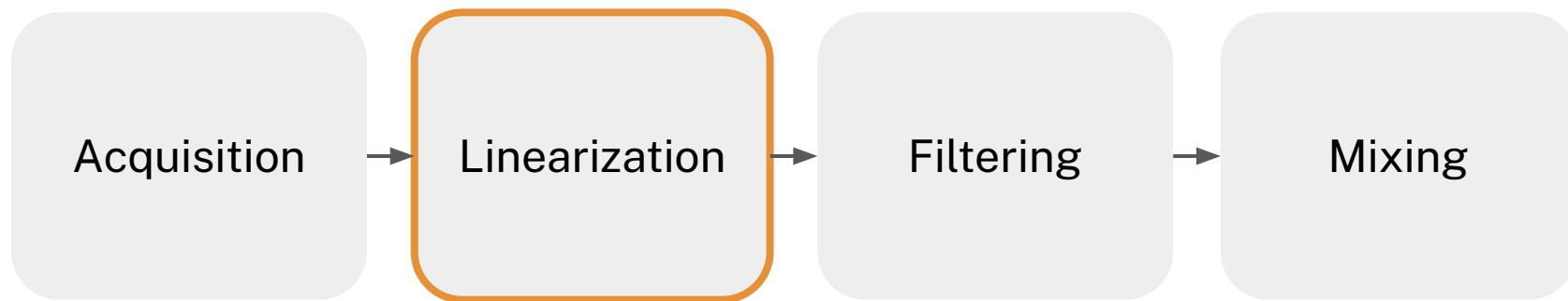
Data curation



Acquisition

	Why?	Examples
Encyclopedia	Educational and maintained with authority and reliability	Wikipedia
Books	High quality, longer textual content, breadth of subjects and topics, educational (e.g. textbooks)	Project Gutenberg, BookCorpus, Books1, Books2, Books3
Academic	Papers, journal articles; highly professional with rigor and domain-specific expertise	arXiv, S2ORC, PubMed
Code	Critical for coding agents	The Stack, Github, BIG-QUERY, StarCoder
Math	Critical for math reasoning	OpenMathText, FineMath
Forums	Question-answering, everyday knowledge and domain-specific expertise	StackExchange, Reddit
General web	Highly diverse and a great amount of breadth	Common Crawl

Data curation



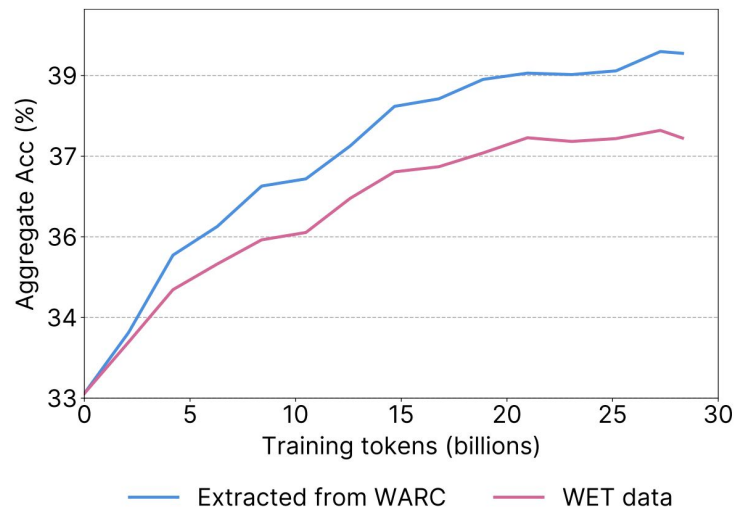
Linearization

HTML, PDF → Plain text

- Remove boilerplate
- Retain LaTeX, code, etc

Example: Common Crawl comes in two formats: WET (text-only, from `htmlparser`) and WARC (all of the raw data).

Plot compares default WET vs. text extracted from WARC w/ `trafilatura`



[FineWeb paper](#)

Linearization

HTML, PDF → Plain text

- Remove boilerplate
- Retain LaTeX, code, etc

Example: OlmoOCR converts PDFs into clean, well-ordered text.

Includes a unit test framework for evaluating the parsing of math equations, tables, multi-column formats, etc.

PDF & LaTeX ground truth

“...represented by the following equation:

$$C_T(u_n^T X_n^{\text{Test}}, \bar{x}^{\text{Test}})$$

”

```
C_T \left( u_n^T X_n^{\text{Test}}, \overline{x}^{\text{Test}} \right)
```

OCR models output

Model A

```
C_{T} \left( u_n^T X_n^{\text{Test}}, \overline{x}^{\text{Test}} \right)
```

Model B

```
C_T \left( u_n^T X_n^{\text{Test}}, \overline{x}^{\text{Test}} \right)
```

“... in the table below:

Model	Size (if known)	Score
moOCR		0.4

”

```
<tr style="font-weight: bold;">  
<th>Model</th>  
<th>Size (if known)</th>  
<th>Score</th>  
</tr>  
<tr>  
<td>moOCR</td>  
<td></td>  
<td>0.4</td>  
</tr>
```

Model A

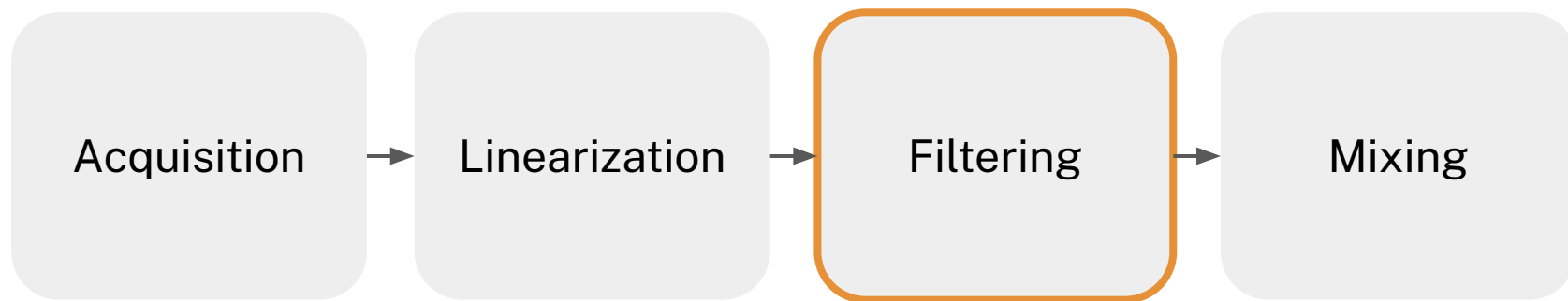
```
<tr style="font-weight: bold;">Model</tr>  
<th style="font-weight: bold;">Size (if known)</th>  
</tr>  
<tr>  
<td>moOCR</td>  
<td></td>  
<td>0.4</td>  
</tr>
```

Model B

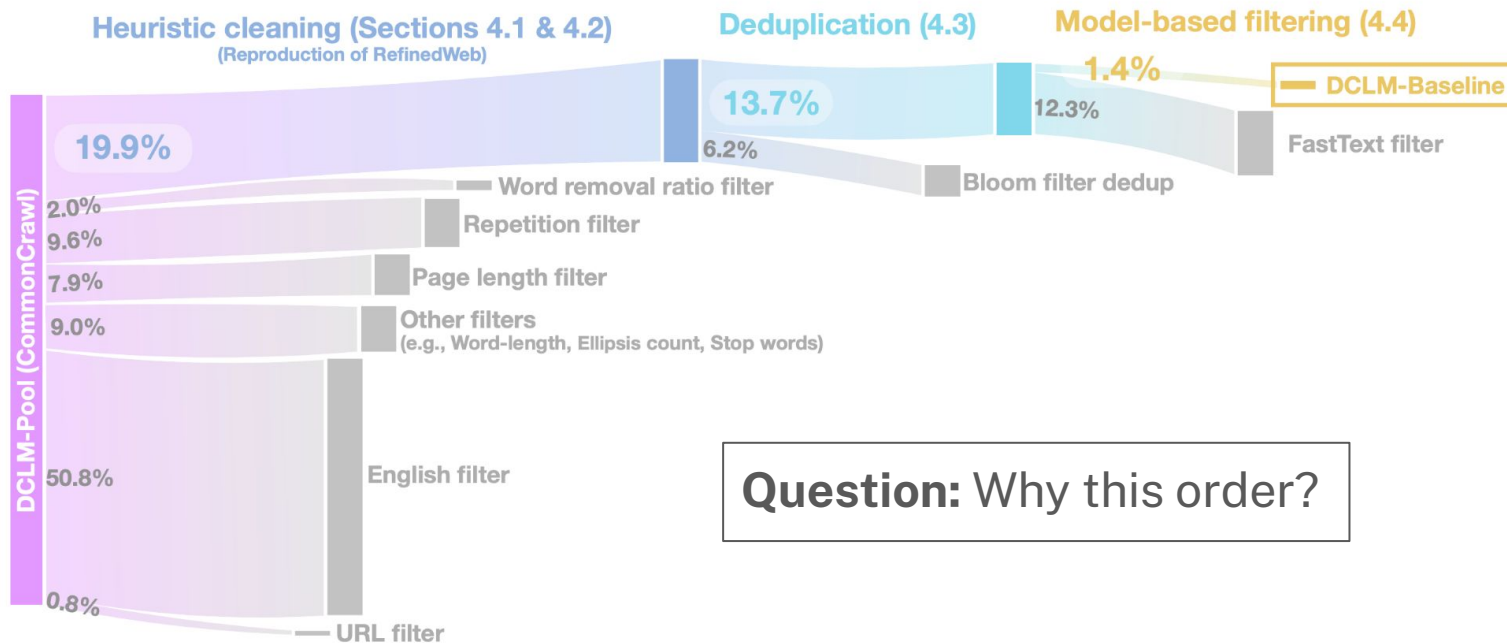
```
<tr style="font-weight: bold;">  
<th>Model</th>  
<th>Size (if known)</th>  
<th>Score</th>  
</tr>  
<tr>  
<td>moOCR</td>  
<td></td>  
<td>0.4</td>  
</tr>
```

[OlmoOCR paper](#)

Data curation



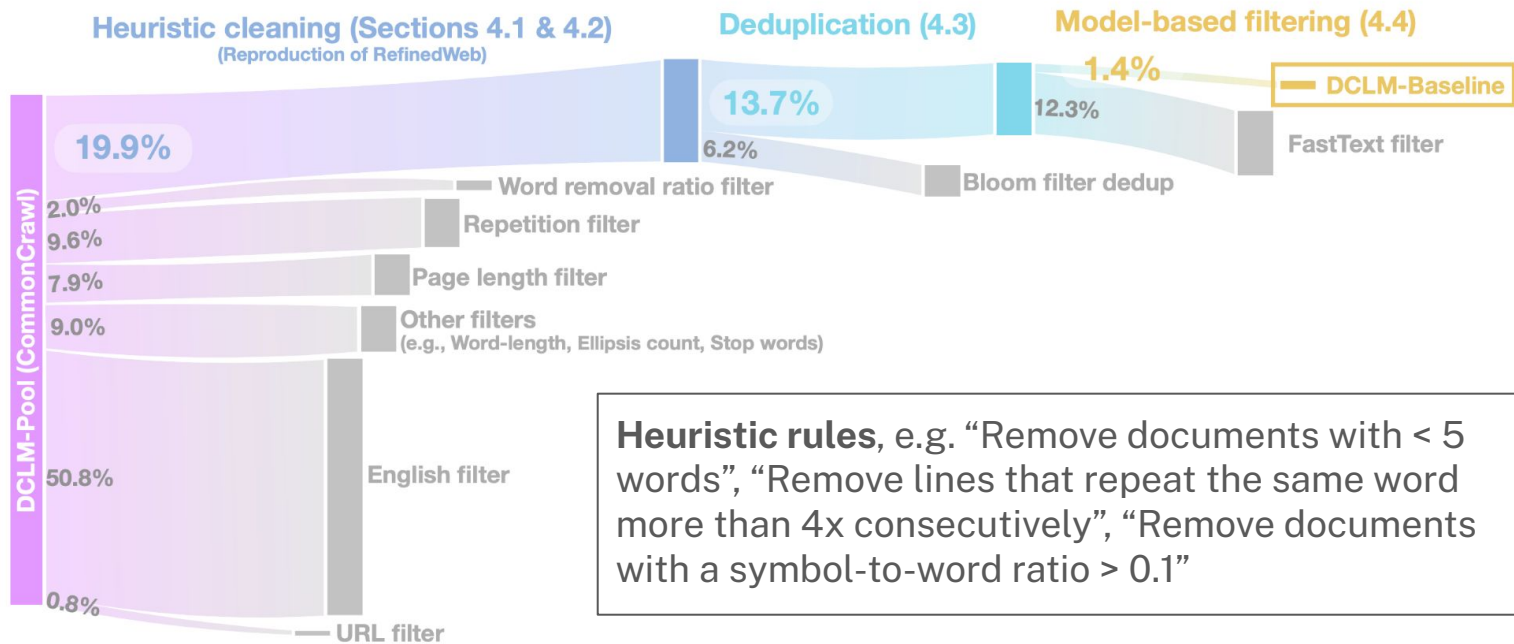
Filtering



Question: Why this order?

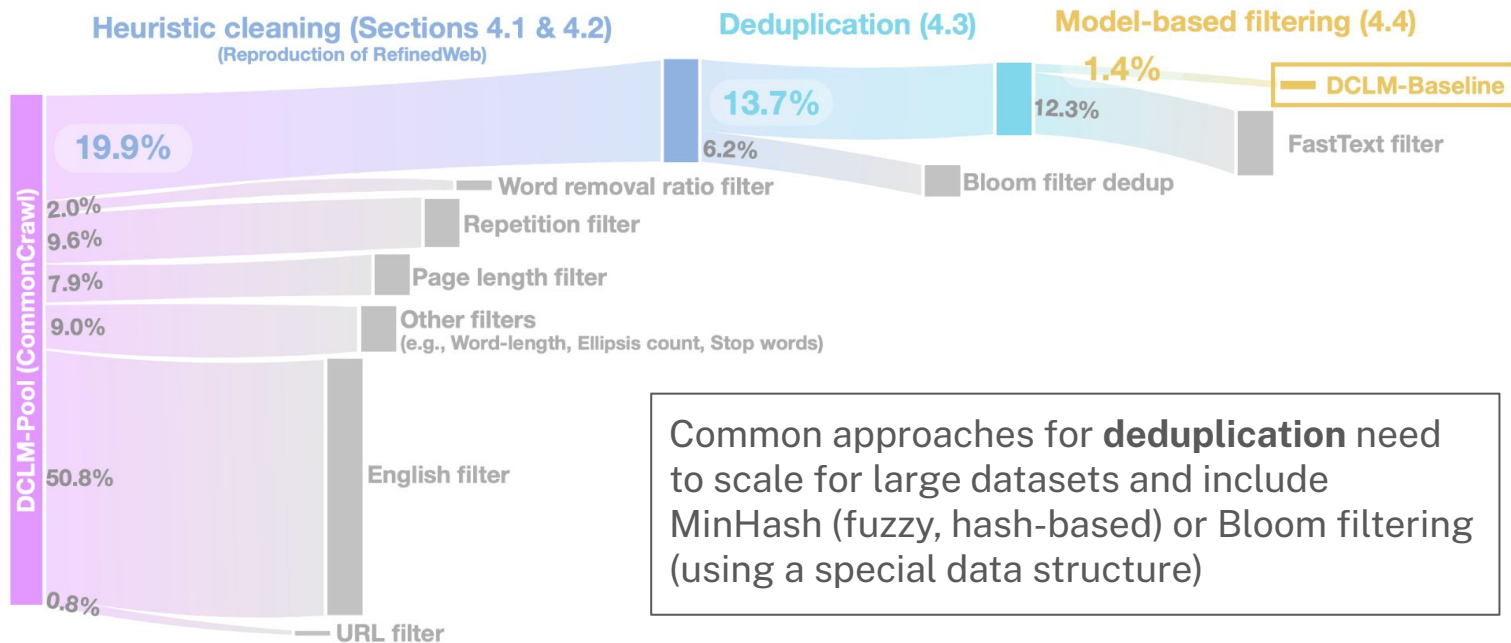
From [DCLM paper](#)

Filtering



From [DCLM paper](#)

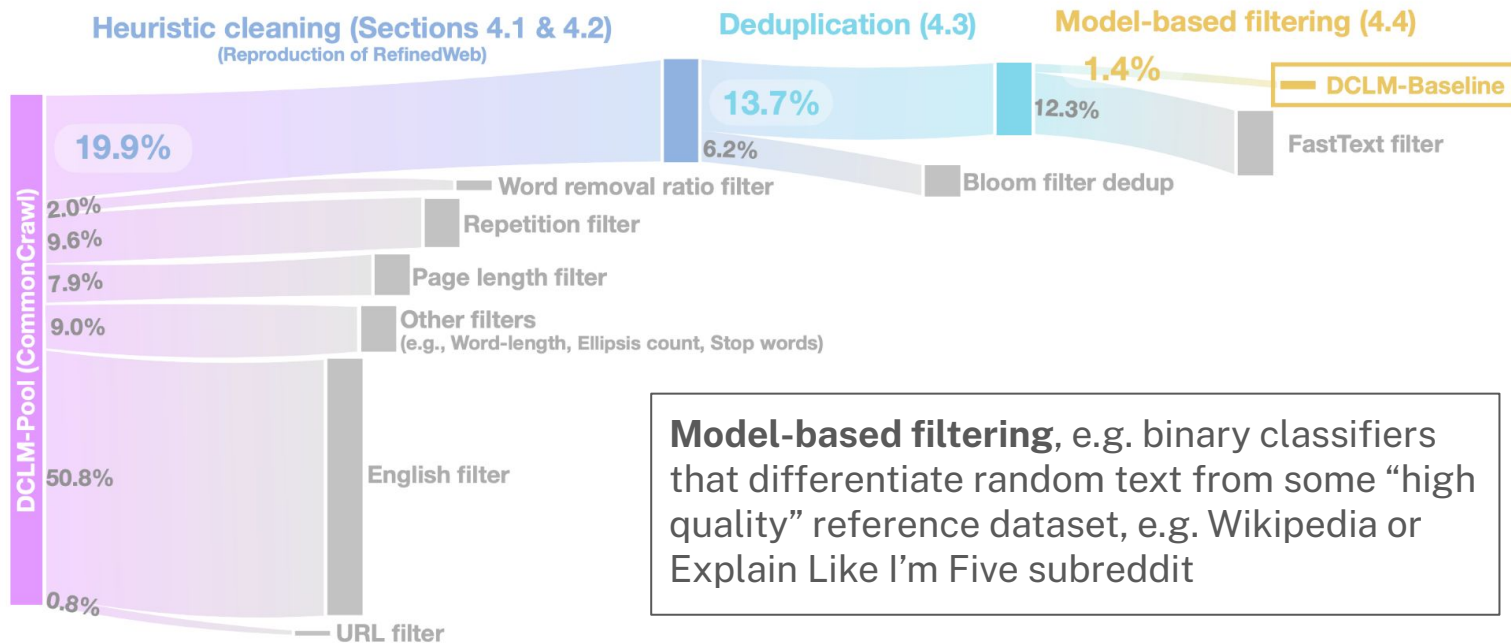
Filtering



Common approaches for **deduplication** need to scale for large datasets and include MinHash (fuzzy, hash-based) or Bloom filtering (using a special data structure)

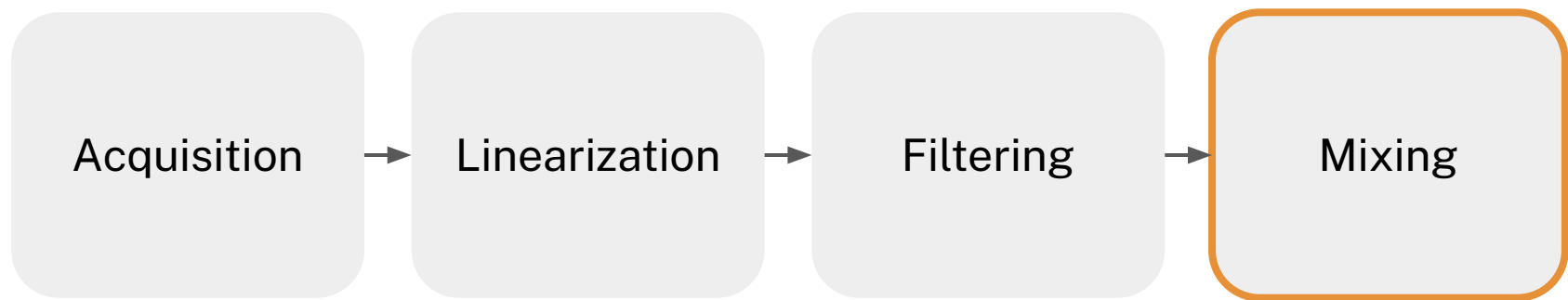
From [DCLM paper](#)

Filtering



From [DCLM paper](#)

Data curation



Mixtures

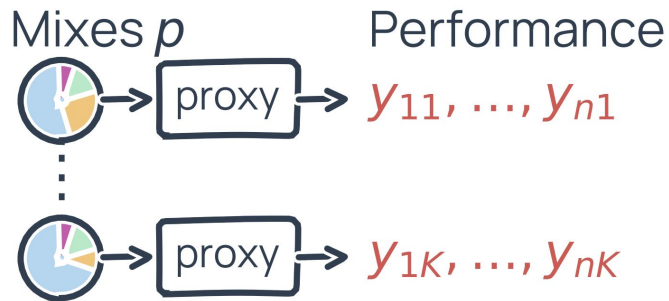
How much of each source should we include in the final mix? Should we upsample any source?

Source	Type	9T Pool		6T Mix	
		Tokens	Docs	Tokens	Docs
Common Crawl	Web pages	8.14T	9.67B	4.51T (76.1%)	3.15B
OLMOOCR science PDFs	Academic documents	972B	101M	805B (13.6%)	83.8M
Stack-Edu (Rebalanced)	GitHub code	137B	167M	409B (6.89%)	526M
arXiv	Papers with LaTeX	21.4B	3.95M	50.8B (0.86%)	9.10M
FineMath 3+	Math web pages	34.1B	21.4M	152B (2.56%)	95.5M
Wikipedia & Wikibooks	Encyclopedic	3.69B	6.67M	2.51B (0.04%)	4.24M
Total		9.31T	9.97B	5.93T (100%)	3.87B

Data mix in the [Olmo 3 paper](#)

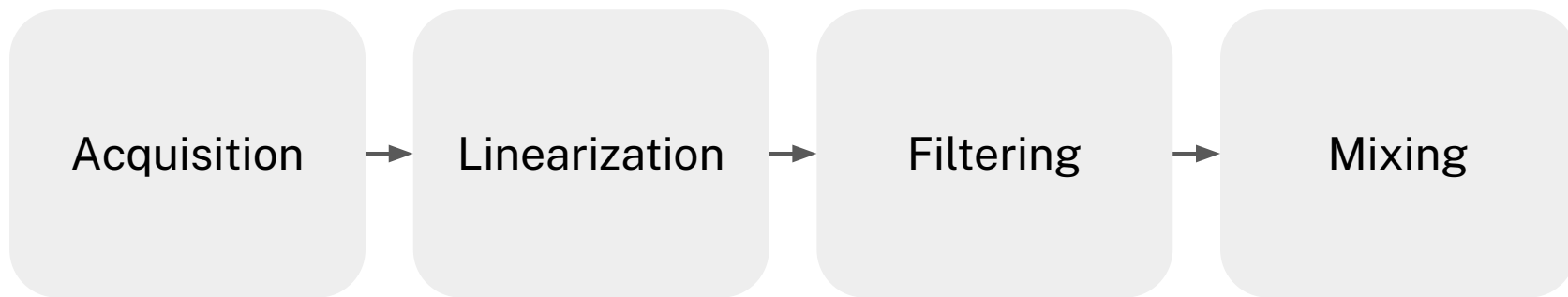
Mixing recipe

1. Classify documents into topics and quality levels
2. Train a swarm of small models (e.g. 30M params), each w/ a different data mix
3. For each eval task, fit a regression model for mixture weights $\rightarrow$ task performance
4. Solve for a mixture that minimizes avg regression-predicted task performance

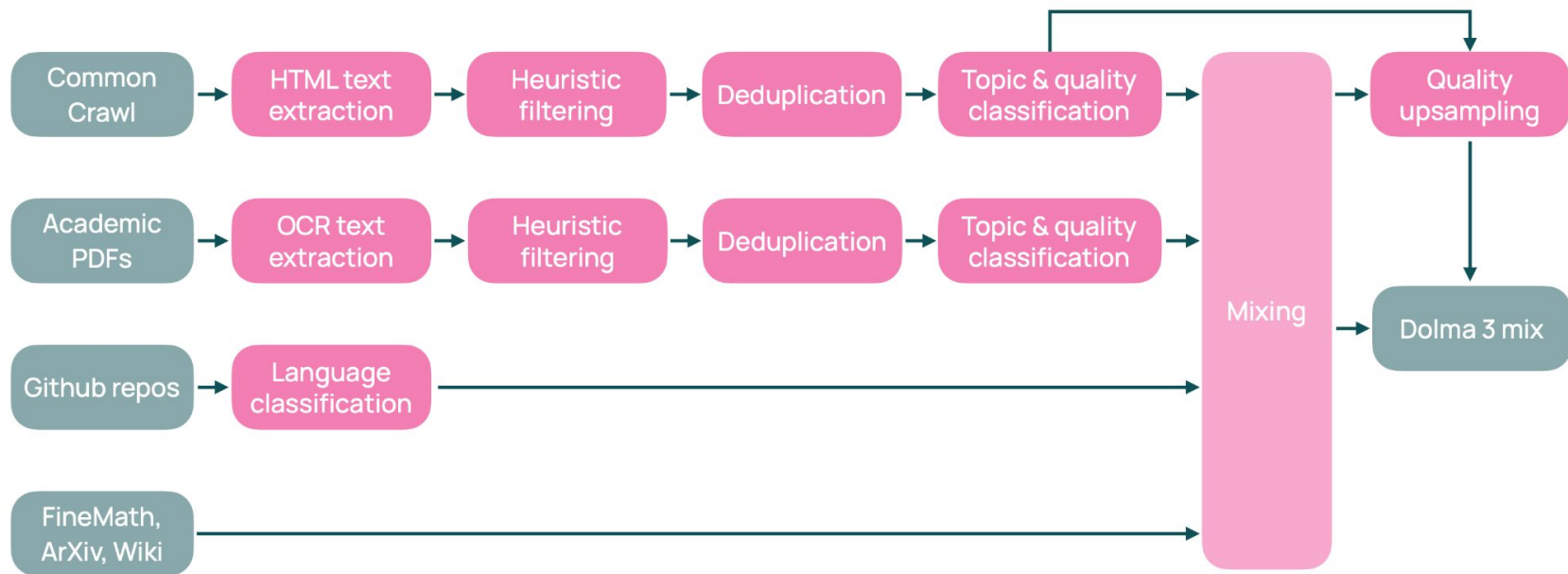


From [the Olmix paper](#)

Data curation



Example: Olmo 3's Dolma 3



From [Olmo 3 paper](#)

Synthetic data

Some say that we are running out of high quality data to train our models!

A proposed solution: use **synthetic data** to augment our dataset

Generator-driven

- Use large LMs to generate data for smaller LMs, e.g. Phi 1.5 ([Li et al. 2023](#))
- Susceptible to model collapse

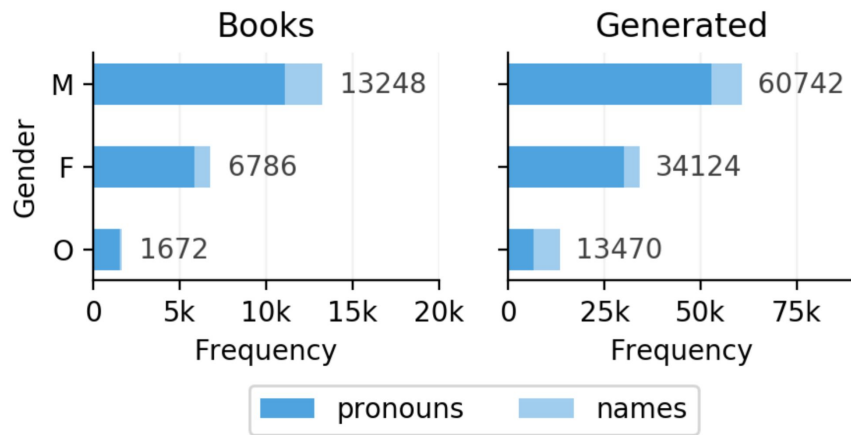
Source rephrasing

- Use an LM to rephrase existing web documents into higher-quality or differently structured formats ([Maini et al. 2024](#), [Nguyen et al. 2025](#))

Data issues: social biases

Language	Percent	Language	Percent
en	89.70%	uk	0.07%
unknown	8.38%	ko	0.06%
de	0.17%	ca	0.04%
fr	0.16%	sr	0.04%
sv	0.15%	id	0.03%
zh	0.13%	cs	0.03%
es	0.13%	fi	0.03%
ru	0.13%	hu	0.03%
nl	0.12%	no	0.03%
it	0.11%	ro	0.03%
ja	0.10%	bg	0.02%
pl	0.09%	da	0.02%
pt	0.09%	sl	0.01%
vi	0.08%	hr	0.01%

From [the Llama 2 paper](#)

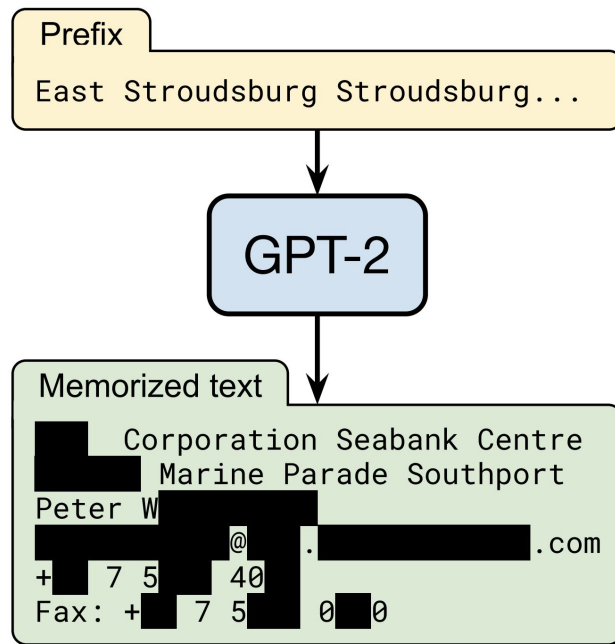


From [Lucy's paper](#)

Data issues: privacy

Models may generate content that they've memorized from their pretraining data.

Sometimes this data is semi-private information!



From [Carlini et al. 2021](#)

Data issues: copyright

- Bartz v. Anthropic, No. 3:24-cv-05417 (N.D. Cal.)
- O’Nan v. Databricks, Inc., Mosaic ML, No. 3:23-cv-01451 (N.D. Cal.)
- Makkai v. Databricks, Inc., Mosaic ML, No. 3:24-cv-02653 (N.D. Cal.)
- Nazemian v. NVIDIA Corp., No. 3:23-cv-01454 (N.D. Cal.)
- Dubus v. NVIDIA Corp., No. 4:24-cv-02655 (N.D. Cal.)
- Kadrey v. Meta Platforms, Inc., No. 3:23-cv-03417 (N.D. Cal.)
- Chabon v. Meta Platforms, Inc., No. 3:23-cv-04663 (N.D. Cal.)
- Huckabee v. Meta Platforms, Inc., No. 3:23-cv-06663 (N.D. Cal.)
- Farnworth v. Meta Platforms, Inc., No. 3:23-cv-06893 (N.D. Cal.)
- J.L. v. Alphabet Inc., No. 3:23-cv-03440-LB (N.D. Cal.) (social media posts)
- In re OpenAI ChatGPT Litigation (transferred to Southern District of New York on April 3, 2025)
- Tremblay v. OpenAI, Inc., No. 3:23-cv-03223 (N.D. Cal.)
- Silverman v. OpenAI, Inc., No. 3:23-cv-03223 (N.D. Cal.)
- Chabon v. OpenAI Inc., No. 3:23-cv-04625 (N.D. Cal.)
- Authors Guild v. OpenAI & Microsoft, No. 1:23-cv-8292 (S.D.N.Y.)
- Alter v. OpenAI & Microsoft, No. 1:23-cv-10211 (S.D.N.Y.)
- Basbanes v. Microsoft Corp., OpenAI, No. 1:24-cv-00084 (S.D.N.Y.)
- Huckabee v. Bloomberg, No. 1:23-cv-11195 (S.D.N.Y.)

Data issues: copyright

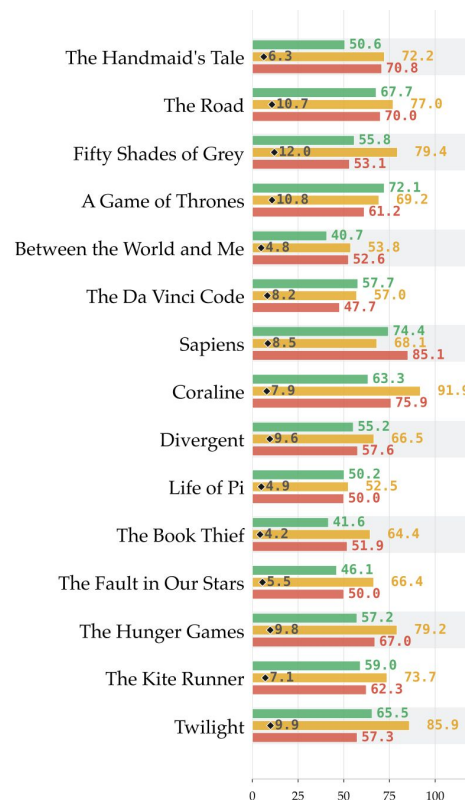
Model developers have created output filters to block verbatim generation of copyrighted text.

However, these filters can be overridden by fine-tuning LMs on other authors' books.

Right: book memorization coverage, or the fraction of words in a test book covered by at least one extracted span containing 5 or more words.

◆ Baseline (Aligned instruction-tuned GPT-4o) ■ Gemini-2.5-Pro finetuned

■ GPT-4o finetuned ■ DeepSeek-V3.1 finetuned



Reminder

Proposal/brainstorming pitch slides due at Sept 22 11:59 pm!

Submit a single slide.

Make sure the slide is publicly viewable by anyone with the link.